Introducción a la programación web, Python + FastAPI

Sistema Centralizado de Avisos de Dominio (SCAD)

Herramientas:

- Python
- `uv` (gestor de dependencias/entorno)
- `FastAPI`
- `SQLModel`
- `SQLite`
- `Jinja2`
- `pytest`
- `HTMX`

Resumen de la arquitectura del proyecto

- Gestor de proyecto: `uv` (Astral) para resolver dependencias, entorno virtual y ejecución.
- Backend: FastAPI con SQLModel (ORM) y SQLite.
- Frontend: HTML, CSS, JavaScript, HTMX con Jinja2 (Markdown a HTML en servidor).
- Testing: `pytest` integrado desde el Día 1 con `httpx` / `TestClient`.
- Cliente Logon: Script de inicio que invoca la API y lanza el navegador predeterminado.

Día 1: Configuración con `uv`, primeras rutas y tests básicos

Bloque 1 (3h): Inicialización con `uv` y fundamentos web

- Objetivos: Inicializar el proyecto con `uv`, entender la arquitectura HTTP/REST y arrancar FastAPI.

- Contenidos:
  1. Introducción a `uv`: comandos `uv init`, `uv add` y `uv run`.
  2. Conceptos HTTP: métodos (GET, POST), códigos de estado y formato JSON.
  3. Estructura inicial del proyecto y primer servidor FastAPI (`main.py`).
  4. Ejecución del servidor con `uv run uvicorn main:app --reload` y exploración de `/docs`.
  5. Primeras pruebas con `pytest`

Introducción a uv (Astral)

- ¿Qué es `uv`? Es una herramienta escrita en Rust, extremadamente rápida, desarrollada por Astral para reemplazar `pip`, `pip-tools`, `virtualenv` y `poetry` en un único binario.
- Ventajas principales:
  - Velocidad: Entre 10x y 100x más rápida que `pip`.
  - Gestión unificada: Crea el entorno virtual (`.venv`), gestiona las versiones de Python y resuelve dependencias en un único archivo lock (`uv.lock`).
  - Aislamiento: Mantiene las librerías del proyecto independientes del sistema operativo.
- Comandos esenciales:
  - `uv init <nombre_proyecto>`: Crea la estructura base del proyecto con un archivo `pyproject.toml`.
    - `uv add <libreria>`: Descarga e instala dependencias en el entorno virtual activo.
    - `uv run <comando>`: Ejecuta un comando o script asegurando que utiliza el entorno virtual del proyecto sin necesidad de "activarlo" manualmente.

Configuración del proyecto con uv

```
# 1. Crear el directorio principal del curso e inicializar el proyecto con uv
mkdir scad
cd scad
uv init --no-package --python 3.12
```

```
# 2. Crear las estructuras internas para profesor y alumno
New-Item -ItemType Directory -Path "profesor\src", "profesor\tests",
"alumno\src", "alumno\tests" -Force

# 3. Crear los archivos __init__.py vacíos para tratarlos como
paquetes de Python
New-Item -ItemType File -Path "profesor\src\__init__.py",
"profesor\tests\__init__.py", "alumno\src\__init__.py",
"alumno\tests\__init__.py" -Force

# 4. (Opcional) Eliminar el main.py que genera uv init en la raíz si
no lo vas a usar
Remove-Item
```

Estructura de directorios del proyecto

```
scad/
├── .venv/                      <-- Entorno virtual único para todo
el proyecto
├── .python-version
├── pyproject.toml              <-- Dependencias globales
├── uv.lock
│
├── profesor/
│   ├── src/
│   │   ├── __init__.py         <-- Importante: convierte 'src' en
paquete
│   │   └── ...
│   └── tests/
│       ├── __init__.py
│       └── ...
│
└── alumno/
    ├── src/
    │   ├── __init__.py         <-- Importante: convierte 'src' en
paquete
    │   ├── main.py
    │   └── ...
    └── tests/
```

```
        ├── __init__.py
        └── ...
```

Modificamos el fichero `pyproject.toml`

```toml
[project]
name = "scad"
version = "0.1.0"
description = "Sistema de control de avisos del directorio"
readme = "README.md"
requires-python = ">=3.12"
dependencies = [
    "fastapi>=0.141.1",
    "uvicorn>=0.53.0",
]

[dependency-groups]
dev = [
    "httpx2>=2.13.0",
    "pytest>=9.1.1",
]

[tool.pytest.ini_options]
pythonpath = "."
```

Fundamentos de la Web y HTTP

- Arquitectura Cliente-Servidor:
    - Cliente: Quien solicita información (un navegador web, un script en Python, `logon.cmd`, Postman).
        - Servidor: Equipo que escucha peticiones, procesa lógica de negocio y responde con datos.
- Anatomía de una Petición HTTP (Request):
    - URL/URI: La dirección del recurso (`http://servidor:8000/api/v1/avisos`).
        - Métodos / Verbos HTTP:
            - `GET`: Consultar / Obtener información (sin modificar nada en el servidor).

        - `POST`: Crear un nuevo recurso.
        - `PUT` / `PATCH`: Modificar un recurso existente.
        - `DELETE`: Eliminar o desactivar un recurso.
    - Cabeceras (Headers): Metadatos de la petición (ej. `Content-Type: application/json`, autenticación).
    - Cuerpo (Body): Datos enviados al servidor (habitualmente en solicitudes `POST` y `PUT`).
- Anatomía de una Respuesta HTTP (Response):
    - Código de Estado (Status Code):
        - `2xx` (Éxito): `200 OK`, `201 Created`, `204 No Content`.
        - `4xx` (Error del Cliente): `400 Bad Request`, `401 Unauthorized`, `404 Not Found`.
        - `5xx` (Error del Servidor): `500 Internal Server Error`.
    - Cuerpo de la respuesta: Habitualmente en formato JSON (JavaScript Object Notation), una estructura clave-valor muy similar a los diccionarios de Python.

Introducción a FastAPI

- ¿Qué es FastAPI? Un *framework* web moderno y de alto rendimiento en Python para construir APIs.
- Características clave:
    - Basado en estándares: Utiliza Pydantic para la validación de datos y OpenAPI para la documentación.
    - Documentación interactiva automática: Genera de forma nativa la interfaz Swagger en `/docs` para probar endpoints sin programar un frontend.
    - Rápido de programar: Minimiza la duplicación de código gracias a los tipos de datos nativos de Python.

Código de la aplicación `.\scad\profesor\src\main.py`

```
from fastapi import FastAPI


# Se crea la instancia principal de la aplicación
```

```
app = FastAPI(
    title="SCAD - Sistema Centralizado de Avisos de Dominio",
    description="API para la gestión y distribución de avisos a
equipos y usuarios de dominio",
    version="0.1.0"
)


# Decorador que asocia el método GET en la ruta raíz "/" con la
función
@app.get("/")
def inicio():
    """
    Ruta de prueba de estado del servidor.
    Devuelve un objeto JSON indicando que el servicio está activo.
    """
    return {
        "estado": "online",
        "sistema": "SCAD",
        "mensaje": "Bienvenido al Sistema Centralizado de Avisos de
Dominio"
    }


# Segunda ruta de comprobación rápida de salud (Health Check)
@app.get("/ping")
def ping():
    """
    Endpoint ligero utilizado por scripts para verificar
disponibilidad.
    """
    return {"status": "ok"}
```

Añadiendo dependencias del proyecto con `uv`, en función del código

```
# desde el directorio scad
uv add fastapi uvicorn
```

Arranque del servidor y demostración

```
# desde el directorio scad
uv run uvicorn profesor.src.main:app --reload
```

- Explicación de los parámetros:
  - `profesor.src.main:app`: Busca el objeto `app` dentro del archivo `main.py` del directorio `scad\profesor\src\min`.
  - `--reload`: Reinicia automáticamente el servidor cada vez que guardamos un cambio en el código.
- Abrir un navegador:
  1. http://127.0.0.1:8000/ -> Ver la respuesta JSON directa.
  2. http://127.0.0.1:8000/docs -> Mostrar la interfaz de Swagger UI
  3.
     2. http://127.0.0.1:8000/redoc -> Mostrar la interfaz de Redocly

Ejercicio práctico para el alumno

Enunciado del Ejercicio

1. Inicializa tu propio proyecto utilizando `uv` con el nombre `scad`.
2. Instala `fastapi` y `uvicorn`
3. Edita el archivo `.\scad\alumno\src\main.py` y añade las siguientes rutas:
   - `GET /version`: Debe devolver un JSON con la versión actual del programa y el nombre de la organización (`{"app": "SCAD", "version": "1.0.0", "organizacion": "MiDominio"}`).
   - `GET /equipo-desarrollo`: Debe devolver un JSON que contenga una lista con los nombres de los miembros del equipo de administración/desarrollo.
   - `GET /info-dominio`: Debe devolver la información simulada del dominio de red (`nombre_dominio`, `servidor_dc` y `estado_dc`).
4. Inicia el servidor y prueba todas las rutas desde la documentación Swagger en `/docs`.

Solución del ejercicio práctico

Código resuelto `.\scad\alumno\src\main.py`

```python
from fastapi import FastAPI


app = FastAPI(
    title="SCAD - Sistema Centralizado de Avisos",
    description="Ejercicio práctico",
    version="1.0.0"
)


@app.get("/")
def inicio():
    return {"mensaje": "Servidor del ejercicio iniciado
correctamente"}


@app.get("/version")
def obtener_version():
    return {
        "app": "SCAD",
        "version": "1.0.0",
        "organizacion": "MiDominio"
    }


@app.get("/equipo-desarrollo")
def obtener_equipo():
    return {
        "equipo": ["Admin General", "Soporte Técnico", "Operaciones
IT"],
        "total_miembros": 3
    }


@app.get("/info-dominio")
def obtener_info_dominio():
    return {
        "nombre_dominio": "empresa.local",
        "servidor_dc": "DC01.empresa.local",
        "estado_dc": "activo"
    }
```

Ejecutando la aplicación y revisa la documentación que publica automáticamente

```
# desde el directorio scad
uv run uvicorn alumno.src.main:app --reload
```

Tests con `pytest`

Para cerrar la sesión, explicaremos cómo los tests aseguran que las rutas funcionan sin necesidad de abrir el navegador manualmente.

Código del archivo de pruebas (tests_main.py)

Crear en la carpeta del proyecto, al mismo nivel que `scr`, el directorio `tests` y dentro el archivo `test_main.py`:

```
from fastapi.testclient import TestClient
from scad_practica_1 import app


# Creamos el cliente de pruebas basado en nuestra app de FastAPI
client = TestClient(app)

def test_ruta_inicio():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json()["mensaje"] == "Bienvenido al Sistema
Centralizado de Avisos de Dominio"


def test_ruta_version():
    response = client.get("/version")
    assert response.status_code == 200
    datos = response.json()
    assert datos["app"] == "SCAD"
    assert datos["version"] == "1.0.0"
    assert "organizacion" in datos


def test_ruta_equipo_desarrollo():
    response = client.get("/equipo-desarrollo")
```

```
    assert response.status_code == 200
    datos = response.json()
    assert isinstance(datos["equipo"], list)
    assert len(datos["equipo"]) == 3


def test_ruta_no_existente():
    # Comprobar que una ruta que no existe devuelve un error 404
    response = client.get("/ruta-falsa")
    assert response.status_code == 404
```

Ejecución de los tests

Tenemos que añadir la nueva dependencia al proyecto

```
# desde el directorio scad
uv add --dev httpx2 pytest
```

Ejecutar las pruebas mediante `uv`:

```
# desde el directorio scad\alumno\tests
uv run pytest -v
```

- Explicación de la salida de `pytest`: Mostrar los verdes (`PASSED`), amarillos (`WARNING`) y rojos (`FAILED`), la velocidad de ejecución y cómo `pytest` valida las respuestas HTTP.

Bloque 2 (3h): Validaciones con `Pydantic` y primeros tests con `pytest`

- Objetivos: Validar datos de entrada/salida e introducir la metodología de testing automatizado usando `TestClient`.
- Contenidos:
  1. Tipos de parámetros HTTP
     - Path Parameters (Parámetros de Ruta): Forman parte de la URL `/avisos/{aviso_id}`. Se usan para identificar un recurso específico.

    - Query Parameters (Parámetros de Consulta): Van al final de la URL tras el símbolo `?` (`/avisos?activo=true&limit=10`). Opcionales o para filtrado.
    - Request Body (Cuerpo de la Petición): JSON enviado por el cliente en peticiones `POST` , `PUT` o `PATCH` . Contiene la información estructurada que se quiere procesar o guardar.
2. Validación de datos con Pydantic (Modelos de solicitud para los avisos).
3. Introducción a `pytest` : estructura de tests, aserciones y el cliente de pruebas de FastAPI (`TestClient`).
4. Ejecución de tests con `uv run pytest` .

Parámetros HTTP

Lo más sencillo es analizar cualquier URL, por lo que tomaremos el ejemplo de una búsqueda con Google

```
https://www.google.com/search?client=firefox-b-
e&channel=entpr&q=salud
```

Analizando sus partes

- `https` : protocolo
- `www` : subdominio
- `google` : dominio
- `.com` : `tld` o dominio de primer nivel
- `/search` : parámetro de ruta (puede haber varios, separados por barras)
- `?client=firefox-b-e&channel=entpr&q=salud` : parámetros de consulta (conjuntos de pares `clave=valor` , separados por `&`)
    - `client=firefox-b-e` : la clave `client` toma en valor `firefox-b-e`
    - `channel=entpr` : la calve `channel` toma el valor `entpr`
    - `q=salud` : la clave `q` toma el valor `salud`

Validación de Datos con Pydantic

- Pydantic es la librería de análisis, conversión y validación de datos basada en `Type Hints` , o definición de tipos, nativos de `Python` que

utiliza `FastAPI` por debajo.

- Ventajas de Pydantic
  - Conversión automática: Si envías `"123"` a un campo `int`, `Pydantic` lo convierte a entero automáticamente.
  - Errores claros (`HTTP 422 Unprocessable Entity`): Si el cliente envía datos con un tipo incorrecto o faltan campos obligatorios, `FastAPI` responde con detalles exactos de lo que ha fallado.
  - Estructura limpia: Heredando de `pydantic.BaseModel` se definen los esquemas de datos (`schemas`).

Testing con `pytest` y `TestClient`

- ¿Por qué hacer testing desde el primer día?
  - Para asegurar que los cambios futuros no rompen la lógica existente.
  - Para no depender de probar manualmente en el navegador cada vez que tocamos código.
- El `TestClient` de FastAPI
  - Basado en `httpx`.
  - Simula llamadas HTTP a la API directamente en memoria, sin necesidad de levantar el servidor `uvicorn` de fondo.
- Aserciones en Python (`assert`):
  - Verificación del código de estado HTTP (`response.status_code == 200`).
  - Verificación de la estructura del JSON devuelto (`response.json()`).

Ejemplo de código

```
from enum import Enum
from typing import Optional

from pydantic import BaseModel, Field
from fastapi import FastAPI, HTTPException, status


app = FastAPI(
    title="SCAD - Sistema Centralizado de Avisos",
```

```python
    description="API para la gestión y distribución de avisos a equipos y usuarios de dominio",
    version="0.2.0"
)


# Enumeración para restringir los tipos de destino permitidos en nuestro dominio
class TipoDestino(str, Enum):
    USUARIO = "usuario"
    EQUIPO = "equipo"
    GRUPO = "grupo"
    OU = "ou"


# Modelo de datos Pydantic para la creación preliminar de un Aviso
class AvisoCrear(BaseModel):
    titulo: str = Field(min_length=3, max_length=100, description="Título del aviso")
    contenido_md: str = Field(description="Contenido del aviso en formato Markdown")
    tipo_destino: TipoDestino = Field(description="Tipo de destinatario en el Active Directory")
    destino: str = Field(description="Nombre del usuario, equipo, grupo u OU destinatario")
    duracion_dias: Optional[int] = Field(default=1, ge=1, le=30, description="Vigencia en días")


# Definición de rutas

# Ejemplo 1: Path Parameter + Query Parameter
@app.get("/avisos/buscar/{aviso_id}")
def buscar_aviso(aviso_id: int, urgente: bool = False):
    """
    Demostración de Path Parameter (aviso_id) y Query Parameter (urgente).
    """
    return {
        "aviso_id": aviso_id,
        "urgente": urgente,
        "mensaje": f"Buscando aviso con ID {aviso_id}"
```

```
    }


# Ejemplo 2: Request Body con Pydantic
@app.post("/avisos/test-validar",
status_code=status.HTTP_201_CREATED)
def probar_creacion_aviso(aviso: AvisoCrear):
    """
    Recibe un objeto JSON en el cuerpo de la petición y lo valida
contra el modelo Pydantic.
    """
    return {
        "mensaje": "Aviso validado y recibido correctamente",
        "datos": aviso,
        "destino_formateado": f"{aviso.tipo_destino.value.upper()}:
{aviso.destino}"
    }
```

```
# desde el directorio scad
uv run uvicorn profesor.src.main:app --reload
```

Lo podemos probar de varias formas

- Desde la documentación que genera automáticamente `FastApi`: `http://127.0.0.1:8000/docs`
- Desde alguna herramienta especializada
- Con `curl`, desde la terminal
- ...

Ejercicio práctico para el alumno

1. Abre tu proyecto del curso.
2. Implementa un `endpoint` `POST /validar-destino` que sirva para que los scripts de validación del dominio comprueben si un destinatario introducido por un operador es válido.
3. El `endpoint` debe recibir un JSON con la siguiente estructura:
    - `tipo_destino`: Cadena de texto (debe ser obligatoriamente uno de estos 4 valores: `"usuario"`, `"equipo"`, `"grupo"`, `"ou"`).
    - `nombre_destino`: Cadena de texto (de longitud mínima 3 caracteres).

4. **Lógica de negocio:**
    - Si el `tipo_destino` no es uno de los 4 permitidos, o si `nombre_destino` contiene caracteres no válidos (por ejemplo, espacios en blanco al inicio/final o longitud < 3), `FastAPI` debe responder con un error.
    - Además, añade una validación personalizada: Si el `tipo_destino` es `"equipo"` pero el `nombre_destino` no empieza por `"PC-"` o `"SRV-"`, debes lanzar una excepción manual `HTTPException` con código `400 Bad Request` y el detalle `"Los nombres de equipo deben empezar por PC- o SRV-"`.
5. Si pasa todas las validaciones, debe responder con HTTP `200 OK` y el mensaje `{"valido": True, "destino": "TIPO:NOMBRE"}`.
6. Escribe una suite de pruebas automatizadas en `tests/test_validaciones.py` usando `pytest` que compruebe los casos exitosos y los casos de error.

Solución del ejercicio práctico

Código `API`

```
from enum import Enum

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field


app = FastAPI(
    title="SCAD - Sistema Centralizado de Avisos",
    description="Ejercicio práctico",
    version="1.0.0"
)


class TipoDestinoEnum(str, Enum):
    USUARIO = "usuario"
    EQUIPO = "equipo"
    GRUPO = "grupo"
    OU = "ou"
```

```python
class SolicitudValidacionDestino(BaseModel):
    tipo_destino: TipoDestinoEnum
    nombre_destino: str = Field(min_length=3, max_length=50)


@app.post("/validar-destino")
def validar_destino(datos: SolicitudValidacionDestino):
    nombre = datos.nombre_destino.strip()

    # Validación adicional de negocio para equipos del dominio
    if datos.tipo_destino == TipoDestinoEnum.EQUIPO:
        if not (nombre.startswith("PC-") or nombre.startswith("SRV-
")):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Los nombres de equipo deben empezar por PC-
o SRV-"
            )

    return {
        "valido": True,
        "destino": f"{datos.tipo_destino.value}:{nombre}"
    }
```

Código `tests`

```python
from fastapi.testclient import TestClient
from alumno.src.main import app


client = TestClient(app)


# 1. Test de éxito para usuario
def test_validar_destino_usuario_exito():
    payload = {
        "tipo_destino": "usuario",
        "nombre_destino": "jgarcia"
    }
    response = client.post("/validar-destino", json=payload)
```

```
    assert response.status_code == 200
    assert response.json() == {
        "valido": True,
        "destino": "usuario:jgarcia"
    }


# 2. Test de éxito para equipo correcto
def test_validar_destino_equipo_exito():
    payload = {
        "tipo_destino": "equipo",
        "nombre_destino": "PC-AULA101"
    }
    response = client.post("/validar-destino", json=payload)
    assert response.status_code == 200
    assert response.json()["destino"] == "equipo:PC-AULA101"


# 3. Test de error de negocio (Equipo con nombre incorrecto)
def test_validar_destino_equipo_nombre_invalido():
    payload = {
        "tipo_destino": "equipo",
        "nombre_destino": "LAPTOP-JUAN"
    }
    response = client.post("/validar-destino", json=payload)
    assert response.status_code == 400
    assert response.json()["detail"] == "Los nombres de equipo deben
empezar por PC- o SRV-"


# 4. Test de error de Pydantic (Tipo de destino no permitido)
def test_validar_destino_tipo_invalido():
    payload = {
        "tipo_destino": "impresora",  # No existe en la enumeración
        "nombre_destino": "IMP-PISO1"
    }
    response = client.post("/validar-destino", json=payload)
    # Pydantic devuelve 422 Unprocessable Entity
    assert response.status_code == 422


# 5. Test de error de Pydantic (Nombre demasiado corto)
```

```python
def test_validar_destino_nombre_corto():
    payload = {
        "tipo_destino": "grupo",
        "nombre_destino": "ab"  # Menos de 3 caracteres
    }
    response = client.post("/validar-destino", json=payload)
    assert response.status_code == 422
```

Día 2: Persistencia con `SQLModel`, `SQLite` y tests de integración

Bloque 3 (3h): Modelado con `SQLModel` y conexión a `SQLite`

- Objetivos: Diseñar la base de datos relacional con `SQLModel` y gestionar la persistencia de datos.
- Contenidos:
  1. Persistencia de datos y `SQLite`

```
- ¿Por qué necesitamos persistencia?
    - Los datos almacenados en memoria (variables de Python) se pierden cuando el servidor `FastAPI` se reinicia o se apaga.
- ¿Qué es `SQLite`?
    - Una base de datos relacional completa, basada en un único archivo plano (ej. `scad.db`).
    - No requiere instalar servicios externos ni servidores separados (ideal para desarrollo y proyectos locales o empotrados).
    - Soporta transacciones `ACID` y lenguaje `SQL` estándar.
```

  2. ¿Qué es un `ORM` y por qué usamos `SQLModel`?
     - `ORM` (Object-Relational Mapping): Herramienta que traduce tablas y filas de una base de datos a clases y objetos de `Python`. Nos permite interactuar con la base de datos sin escribir `SQL` manualmente.
     - ¿Qué es `SQLModel`?
       - Creado por `Tiangolo` (el mismo autor de `FastAPI`).
       - Diseñado específicamente para integrarse de forma fluida con `FastAPI`.

- La ventaja clave: Combina `Pydantic` (validación de `APIs`) y `SQLAlchemy` (el `ORM` más potente de `Python`) en un único modelo de clases. Evita duplicar código al definir modelos para la `API` y modelos para la base de datos.

3. Clases `table=True` y configuración del motor
   - `Field(primary_key=True)`: Indica la columna primaria de la tabla (única por fila, autoincremental por defecto si es de tipo entero).
   - `table=True`: Le indica a `SQLModel` que esa clase representa una tabla real en `SQLite`.
   - `create_engine`: Establece la conexión física entre nuestra aplicación `Python` y el archivo `.db`.
   - `SQLModel.metadata.create_all(engine)`: Analiza todas las clases declaradas e instruye a `SQLite` para crear las tablas en caso de que no existan.

Código de demostración

1. Instalación de dependencias

```
# desde el directorio scad
uv add sqlmodel
```

2. Creación del archivo de base de datos y modelos
   (`.\scad\profesor\src\models.py`)

```
from typing import Optional
from datetime import datetime
from enum import Enum

from sqlmodel import Field, SQLModel, create_engine, Session


# Enumeración para los tipos de destino del dominio
class TipoDestinoEnum(str, Enum):
    USUARIO = "usuario"
    EQUIPO = "equipo"
    GRUPO = "grupo"
    OU = "ou"
```

```python
# Modelo principal: Aviso
class Aviso(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    titulo: str = Field(index=True)
    contenido_md: str
    tipo_destino: TipoDestinoEnum
    destino: str = Field(index=True)  # Indizado para acelerar
búsquedas por usuario/equipo
    fecha_inicio: datetime
    fecha_fin: Optional[datetime] = None
    activo: bool = Field(default=True)
    creado_por: str
    creado_en: datetime = Field(default_factory=datetime.utcnow)
    modificado_por: Optional[str] = None
    modificado_en: Optional[datetime] = None


# Modelo secundario: Registro de auditoría/logs en base de datos
class LogActividad(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    usuario_admin: str
    accion: str  # Ej: "CREAR_AVISO", "DESACTIVAR_AVISO"
    detalle: str


# Configuración de SQLite
sqlite_file_name = "scad.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

# echo=True permite ver las consultas SQL reales que genera SQLModel
en la terminal
engine = create_engine(sqlite_url, echo=True)


def crear_db_y_tablas():
    """Crea las tablas en el archivo scad.db si no existen"""
    SQLModel.metadata.create_all(engine)
```

3. Integración con la aplicación `FastAPI`

```
from contextlib import asynccontextmanager

from fastapi import FastAPI

from profesor.src.models import crear_db_y_tablas


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Código que se ejecuta AL ARRANCAR la aplicación
    crear_db_y_tablas()
    yield
    # Código que se ejecuta AL APAGAR la aplicación (si fuera
necesario)


app = FastAPI(
    title="SCAD - Sistema Centralizado de Avisos del Directorio",
    description="API para la gestión y distribución de avisos a
equipos y usuarios de dominio",
    version="0.3.0",
    lifespan=lifespan
)


@app.get("/")
def inicio():
    return {"mensaje": "Base de datos SQLite conectada con éxito"}
```

Ejercicio práctico para el alumno

1. Añade la librería `sqlmodel` a tu proyecto mediante `uv add sqlmodel`.
2. Instala la extensión `SQLite Viewer` en `VS Code` para poder inspeccionar visualmente la base de datos que se generará.
3. Crea el archivo `models.py` definiendo la clase `Aviso` con los campos necesarios descritos en la teoría:
   - Identificador único (`id`).
   - Título y contenido en `Markdown` (`contenido_md`).

- Tipo de destino (`usuario` , `equipo` , `grupo` , `ou`) y el nombre del destino.
- Fechas de entrada en vigor (`fecha_inicio`) y fin de actividad (`fecha_fin`).
- Estado de activación (`activo`).
- Campos de auditoría: `creado_por` , `creado_en` , `modificado_por` y `modificado_en` .

4. Define el modelo `LogActividad` para guardar el historial de acciones realizadas por los administradores.
5. Inicia el servidor con `uv run uvicorn main:app --reload` y comprueba que se crea automáticamente el archivo `scad.db` en la raíz del proyecto. Abre `scad.db` con la extensión `SQLite Viewer` para verificar la estructura de las tablas.

Código resuelto (`.\scad\alumno\src\models.py`)

```
from datetime import datetime
from enum import Enum
from typing import Optional

from sqlmodel import Field, SQLModel, create_engine


class TipoDestinoEnum(str, Enum):
    USUARIO = "usuario"
    EQUIPO = "equipo"
    GRUPO = "grupo"
    OU = "ou"


class Aviso(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    titulo: str = Field(min_length=3, max_length=100)
    contenido_md: str
    tipo_destino: TipoDestinoEnum
    destino: str
    fecha_inicio: datetime
    fecha_fin: Optional[datetime] = None
    activo: bool = Field(default=True)
```

```
    creado_por: str
    creado_en: datetime = Field(default_factory=datetime.utcnow)
    modificado_por: Optional[str] = None
    modificado_en: Optional[datetime] = None


class LogActividad(SQLModel, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    timestamp: datetime = Field(default_factory=datetime.utcnow)
    usuario_admin: str
    accion: str
    detalle: str


sqlite_file_name = "scad.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=False)


def crear_db_y_tablas():
    SQLModel.metadata.create_all(engine)
```

Tests con `pytest`

Aprenderemos a verificar que la base de datos se crea correctamente y que podemos realizar operaciones básicas sobre los modelos mediante pruebas unitarias.

Código del archivo de pruebas (`.\scad\alumno\tests\test_models.py`)

```
from datetime import datetime, UTC

from sqlmodel import SQLModel, create_engine, Session, select

from alumno.src.models import Aviso, LogActividad, TipoDestinoEnum


def test_creacion_tablas_y_modelos_en_memoria():
    # Usamos una base de datos SQLite en memoria para no alterar la
```

```
BD real durante los tests
    engine_test = create_engine("sqlite:///:memory:")
    SQLModel.metadata.create_all(engine_test)

    with Session(engine_test) as session:
        # 1. Crear un aviso de prueba
        aviso = Aviso(
            titulo="Aviso de Prueba",
            contenido_md="# Titulo\nMensaje en Markdown",
            tipo_destino=TipoDestinoEnum.GRUPO,
            destino="Sistemas",
            fecha_inicio=datetime.now(UTC),
            creado_por="admin_test"
        )
        session.add(aviso)
        session.commit()
        session.refresh(aviso)

        # Comprobar que SQLite asignó automáticamente el id = 1
        assert aviso.id is not None
        assert aviso.id == 1
        assert aviso.activo is True
        assert aviso.destino == "Sistemas"

        # 2. Consultar el aviso recién guardado
        aviso_db = session.exec(select(Aviso).where(Aviso.id ==
1)).first()
        assert aviso_db is not None
        assert aviso_db.titulo == "Aviso de Prueba"


def test_creacion_log_actividad():
    engine_test = create_engine("sqlite:///:memory:")
    SQLModel.metadata.create_all(engine_test)

    with Session(engine_test) as session:
        log = LogActividad(
            usuario_admin="soporte_dom",
            accion="CREAR_AVISO",
            detalle="Se ha creado el aviso ID 1"
        )
        session.add(log)
```

```
        session.commit()
        session.refresh(log)

        assert log.id == 1
        assert log.usuario_admin == "soporte_dom"
```

Ejecución de las pruebas

```
# desde .\scad\alumno\tests
uv run pytest -v
```

Bloque 4 (3h): Operaciones `CRUD` y fixtures de `Pytest` para base de datos

- Objetivos: Implementar las operaciones `CRUD` e introducir el uso de fixtures en `Pytest` usando una `BD` `SQLite` en memoria para no contaminar la base de datos real.
- Contenidos:
  1. Inyección de dependencias para la sesión (`Depends`)
     - ¿Qué es la inyección de dependencias en `FastAPI` ?
       - Mecanismo que permite entregar recursos, como una conexión a la BD, a los `endpoints` solo cuando reciben una petición `HTTP` , garantizando que el recurso se cierre correctamente al finalizar.
     - El patrón `get_session` con `yield` : Asegura que cada petición HTTP obtenga su propia transacción aislada y que la conexión a SQLite se cierre siempre, incluso si ocurre una excepción.

```
def get_session():
    with Session(engine) as session:
        yield session
```

  2. Operaciones `CRUD` con `SQLModel`

```
- `Create` (Crear): `session.add(objeto)` + `session.commit()` +
`session.refresh(objeto)`.
- `Read` (Leer): `session.exec(select(Modelo).where(...)).all()`
(o `.first()`).
- `Update` (Actualizar): Modificar los atributos de un objeto
```

```
recuperado de la BD y ejecutar `session.add(objeto)` +
`session.commit()`.
- `Delete` (Borrar): `session.delete(objeto)` +
`session.commit()`.
- Separación de Modelos (`DTOs`/`Schemas`):
    - `AvisoCreate`: Datos requeridos para crear (sin `id` ni
fechas de auditoría).
    - `AvisoUpdate`: Campos opcionales para modificaciones
parciales (`PATCH`).
    - `Aviso` (Modelo de BD): Incluye `id`, `creado_en`, etc.
```

3. Testing con Base de Datos y `pytest.fixture`

```
- El peligro de testear sobre la BD de producción/desarrollo
(`scad.db`): Los tests dejan datos "basura" o pueden fallar si
dependen del orden de ejecución.
- ¿Qué es una `fixture` en `pytest`? Una función que prepara un
entorno o recurso antes de ejecutar un test y limpia todo al
finalizar.
- Sobrescribir dependencias (`app.dependency_overrides`):
Permite indicarle a `FastAPI` que, durante los tests, reemplace
la base de datos real `scad.db` por una base de datos SQLite
efímera en memoria (`sqlite:///:memory:`).
```

Código de demostración

Definición de esquemas `DTOs` (`models.py`)

Añadimos a nuestro `models.py` las clases auxiliares para la entrada y actualización de avisos:

```python
from enum import Enum
from datetime import datetime, UTC
from typing import Optional

from sqlmodel import Field, SQLModel, create_engine, Session


class TipoDestinoEnum(str, Enum):
    USUARIO = "usuario"
    EQUIPO = "equipo"
    GRUPO = "grupo"
```

```
    OU = "ou"


# Modelo base de campos comunes
class AvisoBase(SQLModel):
    titulo: str = Field(min_length=3, max_length=100)
    contenido_md: str
    tipo_destino: TipoDestinoEnum
    destino: str
    fecha_inicio: datetime
    fecha_fin: Optional[datetime] = None
    activo: bool = True


# Modelo de datos para la BD (tabla real)
class Aviso(AvisoBase, table=True):
    id: Optional[int] = Field(default=None, primary_key=True)
    creado_por: str
    creado_en: datetime = Field(default_factory=lambda :
datetime(UTC))
    modificado_por: Optional[str] = None
    modificado_en: Optional[datetime] = None


# DTO para crear (recibido en el cliente)
class AvisoCreate(AvisoBase):
    pass


# DTO para actualizar (campos opcionales)
class AvisoUpdate(SQLModel):
    titulo: Optional[str] = None
    contenido_md: Optional[str] = None
    tipo_destino: Optional[TipoDestinoEnum] = None
    destino: Optional[str] = None
    fecha_inicio: Optional[datetime] = None
    fecha_fin: Optional[datetime] = None
    activo: Optional[bool] = None


# Configuración del motor de BD
sqlite_file_name = "scad.db"
```

```
sqlite_url = f"sqlite:///{sqlite_file_name}"
engine = create_engine(sqlite_url, echo=False)


def crear_db_y_tablas():
    SQLModel.metadata.create_all(engine)


# Inyección de dependencia para obtener sesión de BD
def get_session():
    with Session(engine) as session:
        yield session
```

Implementación de las rutas CRUD

Crea las operaciones CRUD completas, en `scad\profesor\src\main.py`

```
from contextlib import asynccontextmanager
from datetime import datetime
from typing import List, Optional

from fastapi import FastAPI, Depends, HTTPException, status
from sqlmodel import Session, select

from profesor.src.models import Aviso, AvisoCreate, AvisoUpdate,
crear_db_y_tablas, get_session


@asynccontextmanager
async def lifespan(app: FastAPI):
    crear_db_y_tablas()
    yield


app = FastAPI(
    title="SCAD - Sistema Centralizado de Avisos del Directorio",
    description="API para la gestión y distribución de avisos a
equipos y usuarios de dominio",
    version="0.4.0",
    lifespan=lifespan
)
```

```python
# 1. CREAR AVISO (POST)
@app.post("/avisos", response_model=Aviso,
status_code=status.HTTP_201_CREATED)
def crear_aviso(aviso_in: AvisoCreate, session: Session =
Depends(get_session)):
    # Simulamos un usuario autenticado por ahora
    usuario_actual = "operador_admin"

    nuevo_aviso = Aviso.model_validate(
        aviso_in,
        update={"creado_por": usuario_actual}
    )
    session.add(nuevo_aviso)
    session.commit()
    session.refresh(nuevo_aviso)
    return nuevo_aviso


# 2. LISTAR AVISOS CON FILTROS (GET)
@app.get("/avisos", response_model=List[Aviso])
def listar_avisos(
    destino: Optional[str] = None,
    solo_activos: bool = True,
    session: Session = Depends(get_session)
):
    query = select(Aviso)
    if solo_activos:
        query = query.where(Aviso.activo == True)
    if destino:
        query = query.where(Aviso.destino == destino)

    avisos = session.exec(query).all()
    return avisos


# 3. OBTENER UN AVISO POR ID (GET)
@app.get("/avisos/{aviso_id}", response_model=Aviso)
def obtener_aviso(aviso_id: int, session: Session =
Depends(get_session)):
    aviso = session.get(Aviso, aviso_id)
```

```
    if not aviso:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"No se ha encontrado el aviso con ID {aviso_id}"
        )
    return aviso


# 4. ACTUALIZAR PARCIALMENTE UN AVISO (PATCH)
@app.patch("/avisos/{aviso_id}", response_model=Aviso)
def actualizar_aviso(
    aviso_id: int,
    aviso_update: AvisoUpdate,
    session: Session = Depends(get_session)
):
    aviso_db = session.get(Aviso, aviso_id)
    if not aviso_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"No se ha encontrado el aviso con ID {aviso_id}"
        )

    # Extraer solo los campos enviados en el JSON
    datos_actualizar = aviso_update.model_dump(exclude_unset=True)
    for clave, valor in datos_actualizar.items():
        setattr(aviso_db, clave, valor)

    aviso_db.modificado_por = "operador_admin"
    aviso_db.modificado_en = datetime.now(UTC)

    session.add(aviso_db)
    session.commit()
    session.refresh(aviso_db)
    return aviso_db


# 5. ELIMINAR UN AVISO (DELETE)
@app.delete("/avisos/{aviso_id}",
status_code=status.HTTP_204_NO_CONTENT)
def eliminar_aviso(aviso_id: int, session: Session =
Depends(get_session)):
    aviso_db = session.get(Aviso, aviso_id)
```

```
    if not aviso_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"No se ha encontrado el aviso con ID {aviso_id}"
        )

    session.delete(aviso_db)
    session.commit()
    return None
```

Ejercicio práctico para el alumno

1. Incorpora las rutas CRUD (`POST` , `GET` , `GET /{id}` , `PATCH /{id}` , `DELETE /{id}`) a tu aplicación `scad-project` .
2. Prueba las rutas manualmente usando la interfaz automática `Swagger UI` interactiva en la URL: `http://127.0.0.1:8000/docs` .
   - Crea un par de avisos dirigidos a diferentes destinos (ej. `usuario:jgarcia` , `equipo:PC-AULA101`).
   - Consulta la lista filtrando por `destino` .
   - Desactiva un aviso utilizando el método `PATCH` .
3. Crea un archivo `conftest.py` en la carpeta `tests\` para definir las **fixtures** globales de `pytest` utilizando una base de datos SQLite en memoria (`sqlite:///:memory:`).

- Crea el archivo `tests\test_crud_avisos.py` con pruebas automatizadas que verifiquen el ciclo de vida completo de un aviso (Creación -> Consulta -> Edición -> Eliminación).

Código resuelto

Configuración de `fixtures` globales: `tests\conftest.py`

El módulo `conftest.py` es reconocido automáticamente por `pytest` para compartir *fixtures* entre todos los archivos de prueba.

```
from fastapi.testclient import TestClient
import pytest
from sqlmodel import SQLModel, Session, create_engine
from sqlmodel.pool import StaticPool
```

```
from alumno.src.main import app
from alumno.src.models import get_session


@pytest.fixture(name="session")
def session_fixture():
    # Base de datos SQLite en memoria única para el test
    engine_test = create_engine(
        "sqlite://",
        connect_args={"check_same_thread": False},
        poolclass=StaticPool
    )
    SQLModel.metadata.create_all(engine_test)
    with Session(engine_test) as session:
        yield session


@pytest.fixture(name="client")
def client_fixture(session: Session):
    # Sobrescribimos la dependencia get_session para usar la BD en
memoria de los tests
    def get_session_override():
        return session

    app.dependency_overrides[get_session] = get_session_override
    client = TestClient(app)
    yield client
    app.dependency_overrides.clear()
```

Tests de integración del `CRUD` : `tests\test_crud_avisos.py`

Gracias a las fixtures declaradas en `conftest.py` , las funciones de prueba simplemente reciben `client` como argumento.

```
from datetime import datetime, UTC


def test_crear_y_leer_aviso(client):
    # 1. Crear un aviso via POST
    payload = {
```

```
        "titulo": "Mantenimiento Servidor AD",
        "contenido_md": "El servidor se reiniciara a las 22:00",
        "tipo_destino": "equipo",
        "destino": "SRV-DC01",
        "fecha_inicio": "2026-03-30T22:00:00Z",
        "activo": True
    }
    response = client.post("/avisos", json=payload)
    assert response.status_code == 201
    datos = response.json()
    assert datos["id"] is not None
    assert datos["titulo"] == "Mantenimiento Servidor AD"
    aviso_id = datos["id"]

    # 2. Obtener el aviso recién creado via GET
    response_get = client.get(f"/avisos/{aviso_id}")
    assert response_get.status_code == 200
    assert response_get.json()["destino"] == "SRV-DC01"


def test_actualizar_aviso_patch(client):
    # Crear un aviso inicial
    payload = {
        "titulo": "Aviso Temporal",
        "contenido_md": "Contenido inicial",
        "tipo_destino": "usuario",
        "destino": "mlopez",
        "fecha_inicio": "2026-03-30T10:00:00Z"
    }
    res_crear = client.post("/avisos", json=payload)
    aviso_id = res_crear.json()["id"]

    # Desactivar el aviso via PATCH
    res_patch = client.patch(f"/avisos/{aviso_id}", json={"activo":
False})
    assert res_patch.status_code == 200
    assert res_patch.json()["activo"] is False
    assert res_patch.json()["modificado_por"] == "operador_admin"


def test_eliminar_aviso_delete(client):
    # Crear aviso
```

```
    payload = {
        "titulo": "Aviso a borrar",
        "contenido_md": "Borrar pronto",
        "tipo_destino": "grupo",
        "destino": "Ventas",
        "fecha_inicio": "2026-03-30T10:00:00Z"
    }
    res_crear = client.post("/avisos", json=payload)
    aviso_id = res_crear.json()["id"]

    # Eliminar aviso
    res_del = client.delete(f"/avisos/{aviso_id}")
    assert res_del.status_code == 204

    # Verificar que ya no existe (404)
    res_get = client.get(f"/avisos/{aviso_id}")
    assert res_get.status_code == 404
```

Día 3: Seguridad, auditoría y `logging`

Bloque 5 (3h): Autenticación de administradores y tests de seguridad

- Objetivos: Proteger las operaciones de administración mediante `HTTP Basic Auth` y validar los permisos con tests.
- Contenidos:
  1. Seguridad y Autenticación en entornos de dominio
     - Diferencia entre Autenticación y Autorización:
       - Autenticación (¿Quién eres?): Verificar la identidad de quien realiza la petición.
         - Autorización (¿Qué puedes hacer?): Comprobar si esa identidad tiene permisos para realizar una acción específica (ej: un script de logon cliente solo lee avisos; un operador de red puede crear o borrar avisos).
     - `HTTP Basic Authentication`:
       - Protocolo estándar sencillo integrado en el encabezado de las peticiones HTTP: `Authorization: Basic`

`<base64(usuario:password)>`.
        - Útil para autenticación rápida de `APIs` de administración en redes locales/intranet sobre `HTTPS`.
2. Módulo `fastapi.security`
    - `HTTPBasic` y `HTTPBasicCredentials`:
        - `FastAPI` provee utilidades integradas que extraen y decodifican automáticamente el encabezado `Authorization`.
        - Si la petición no incluye credenciales, `FastAPI` responde automáticamente con un código **`401 Unauthorized`** y el encabezado `WWW-Authenticate: Basic`, indicando al cliente/navegador que debe identificarse.
    - Comparación de contraseñas de forma segura:
        - Uso de `secrets.compare_digest` para evitar ataques de tiempo, `timing attacks`, al validar contraseñas o tokens.
3. Inyección de Dependencias de Seguridad
    - Protección declarativa de rutas:
        - En lugar de repetir el código de validación de contraseña dentro de cada función de ruta, creamos una dependencia `obtener_usuario_admin_actual`.
        - Cualquier ruta que declare `usuario: str = Depends(obtener_usuario_admin_actual)` quedará automáticamente protegida.
        - Además, esta función devuelve el nombre del usuario autenticado, que se utiliza directamente en los registros de auditoría (`creado_por`, `modificado_por`).

Código de demostración

Paso 1: Dependencia de Autenticación (`profesor\src\auth.py` o dentro de `main.py`)

Veamos cómo implementar una función de seguridad que valide las credenciales contra el dominio, simulado o configurado:

```python
import secrets

from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials


security = HTTPBasic()

# Usuarios administradores simulados del dominio (en producción conectaríamos con Active Directory / LDAP)
ADMIN_USERS = {
    "admin_dom": "P@ssw0rd2026",
    "soporte_dom": "AdminSCAD123"
}


def obtener_usuario_admin_actual(credentials: HTTPBasicCredentials = Depends(security)) -> str:
    """
    Dependencia de seguridad que valida la autenticación HTTP Basic.
    Devuelve el nombre del usuario autenticado si es correcto, o lanza HTTP 401.
    """
    usuario_correcto = False
    if credentials.username in ADMIN_USERS:
        password_almacenada = ADMIN_USERS[credentials.username]
        # Compare_digest evita ataques de temporización al comparar cadenas
        usuario_correcto = secrets.compare_digest(
            credentials.password.encode("utf-8"),
            password_almacenada.encode("utf-8")
        )

    if not usuario_correcto:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Credenciales de administración incorrectas o no proporcionadas",
            headers={"WWW-Authenticate": "Basic"},
        )
```

```
    return credentials.username
```

Paso 2: Protección de Rutas en `profesor\src\main.py`

Separamos el acceso público de la administración protegida:

```
from typing import List, Optional

from fastapi import FastAPI, Depends, status
from sqlmodel import Session, select

from auth import obtener_usuario_admin_actual
from models import Aviso, AvisoCreate, AvisoUpdate, get_session


app = FastAPI(title="SCAD - Módulo de Seguridad", version="0.5.0")


# 1. RUTA PÚBLICA (Acceso de lectura para clientes de dominio / logon.cmd)
@app.get("/avisos", response_model=List[Aviso])
def listar_avisos_publicos(
    destino: Optional[str] = None,
    session: Session = Depends(get_session)
):
    query = select(Aviso).where(Aviso.activo == True)
    if destino:
        query = query.where(Aviso.destino == destino)
    return session.exec(query).all()


# 2. RUTA PROTEGIDA: Crear Aviso (Requiere autenticación de admin)
@app.post("/avisos", response_model=Aviso, status_code=status.HTTP_201_CREATED)
def crear_aviso_protegido(
    aviso_in: AvisoCreate,
    usuario_admin: str = Depends(obtener_usuario_admin_actual),  # <--- Inyección de seguridad
    session: Session = Depends(get_session)
):
```

```
    nuevo_aviso = Aviso.model_validate(
        aviso_in,
        update={"creado_por": usuario_admin}  # Se asigna
automáticamente el usuario autenticado
    )
    session.add(nuevo_aviso)
    session.commit()
    session.refresh(nuevo_aviso)
    return nuevo_aviso


# 3. RUTA PROTEGIDA: Actualizar Aviso (PATCH)
@app.patch("/avisos/{aviso_id}", response_model=Aviso)
def actualizar_aviso_protegido(
    aviso_id: int,
    aviso_update: AvisoUpdate,
    usuario_admin: str = Depends(obtener_usuario_admin_actual),
    session: Session = Depends(get_session)
):
    aviso_db = session.get(Aviso, aviso_id)
    if not aviso_db:
        raise HTTPException(status_code=404, detail="Aviso no
encontrado")

    datos = aviso_update.model_dump(exclude_unset=True)
    for clave, valor in datos.items():
        setattr(aviso_db, clave, valor)

    aviso_db.modificado_por = usuario_admin
    session.add(aviso_db)
    session.commit()
    session.refresh(aviso_db)
    return aviso_db
```

Ejercicio práctico para el alumno

1. Edita el archivo `main.py` de tu proyecto (`alumno\src\main.py`), o crea un módulo nuevo.
2. Añade la función de autenticación `obtener_usuario_admin_actual` usando `HTTPBasic` de `FastAPI` .

3. Configura las siguientes reglas de acceso:
    - `GET /avisos`: Acceso libre y público (sin autenticación).
    - `POST /avisos`, `PATCH /avisos/{id}` y `DELETE /avisos/{id}`: Exigen autenticación HTTP Basic obligatoria.
4. Asegúrate de que al crear o editar un aviso, el campo `creado_por` o `modificado_por` tome el valor devuelto por la función de autenticación de forma dinámica.
5. Comprueba en `Swagger UI` cómo aparece el botón `Authorize` en la esquina superior derecha para autenticarse con usuario y contraseña.
6. Escribe en `alumno\tests\test_seguridad.py` una batería de pruebas de seguridad para verificar los accesos autorizados y denegados.

Código resuelto

Vamos a crear la función de autorización en el `main.py`, por simplicidad.

```
from contextlib import asynccontextmanager
from datetime import datetime, UTC
from enum import Enum
import secrets
from typing import List, Optional

from fastapi import FastAPI, Depends, HTTPException, status
from fastapi.security import HTTPBasic, HTTPBasicCredentials
from pydantic import BaseModel, Field
from sqlmodel import Session, select

from alumno.src.models import Aviso, AvisoCreate, AvisoUpdate,
crear_db_y_tablas, get_session


security = HTTPBasic()

# Usuarios administradores simulados del dominio (en producción
conectaríamos con Active Directory / LDAP)
ADMIN_USERS = {
    "admin_dom": "P@ssw0rd2026",
    "soporte_dom": "AdminSCAD123"
}
```

```python
def obtener_usuario_admin_actual(credentials: HTTPBasicCredentials =
Depends(security)) -> str:
    """
    Dependencia de seguridad que valida la autenticación HTTP Basic.
    Devuelve el nombre del usuario autenticado si es correcto, o
lanza HTTP 401.
    """
    usuario_correcto = False
    print('*' * 50)
    print('comprobando si la solicitud llega con crdenciales')
    print(f'{credentials.username=}')
    if credentials.username in ADMIN_USERS:
        password_almacenada = ADMIN_USERS[credentials.username]
        # Compare_digest evita ataques de temporización al comparar
cadenas
        usuario_correcto = secrets.compare_digest(
            credentials.password.encode("utf-8"),
            password_almacenada.encode("utf-8")
        )

    if not usuario_correcto:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail="Credenciales de administración incorrectas o no
proporcionadas",
            headers={"WWW-Authenticate": "Basic"},
        )

    return credentials.username


@asynccontextmanager
async def lifespan(app: FastAPI):
    # Código que se ejecuta AL ARRANCAR la aplicación
    crear_db_y_tablas()
    yield
    # Código que se ejecuta AL APAGAR la aplicación (si fuera
necesario)


app = FastAPI(
```

```
    title="SCAD - Sistema Centralizado de Avisos",
    description="Ejercicio práctico",
    version="5.0.0",
    lifespan=lifespan
)


class TipoDestinoEnum(str, Enum):
    USUARIO = "usuario"
    EQUIPO = "equipo"
    GRUPO = "grupo"
    OU = "ou"


class SolicitudValidacionDestino(BaseModel):
    tipo_destino: TipoDestinoEnum
    nombre_destino: str = Field(min_length=3, max_length=50)


@app.get("/")
def inicio():
    return {"mensaje": "Servidor del ejercicio iniciado
correctamente"}


@app.get("/version")
def obtener_version():
    return {
        "app": "SCAD",
        "version": "5.0.0",
        "organizacion": "MiDominio"
    }


@app.get("/equipo-desarrollo")
def obtener_equipo():
    return {
        "equipo": ["Admin General", "Soporte Técnico", "Operaciones
IT"],
        "total_miembros": 3
    }
```

```python
@app.get("/info-dominio")
def obtener_info_dominio():
    return {
        "nombre_dominio": "empresa.local",
        "servidor_dc": "DC01.empresa.local",
        "estado_dc": "activo"
    }


@app.post("/validar-destino")
def validar_destino(datos: SolicitudValidacionDestino):
    nombre = datos.nombre_destino.strip()

    # Validación adicional de negocio para equipos del dominio
    if datos.tipo_destino == TipoDestinoEnum.EQUIPO:
        if not (nombre.startswith("PC-") or nombre.startswith("SRV-
")):
            raise HTTPException(
                status_code=status.HTTP_400_BAD_REQUEST,
                detail="Los nombres de equipo deben empezar por PC-
o SRV-"
            )

    return {
        "valido": True,
        "destino": f"{datos.tipo_destino.value}:{nombre}"
    }


# 1. CREAR AVISO (POST)
@app.post("/avisos", response_model=Aviso,
status_code=status.HTTP_201_CREATED)
def crear_aviso_protegido(
    aviso_in: AvisoCreate,
    usuario_admin: str = Depends(obtener_usuario_admin_actual),  #
<--- Inyección de seguridad
    session: Session = Depends(get_session)
):
    nuevo_aviso = Aviso.model_validate(
        aviso_in,
        update={"creado_por": usuario_admin}  # Se asigna
```

```
automáticamente el usuario autenticado
    )
    session.add(nuevo_aviso)
    session.commit()
    session.refresh(nuevo_aviso)
    return nuevo_aviso


# 2. LISTAR AVISOS CON FILTROS (GET)
@app.get("/avisos", response_model=List[Aviso])
def listar_avisos(
    destino: Optional[str] = None,
    solo_activos: bool = True,
    session: Session = Depends(get_session)
):
    query = select(Aviso)
    if solo_activos:
        query = query.where(Aviso.activo == True)
    if destino:
        query = query.where(Aviso.destino == destino)

    avisos = session.exec(query).all()
    return avisos


# 3. OBTENER UN AVISO POR ID (GET)
@app.get("/avisos/{aviso_id}", response_model=Aviso)
def obtener_aviso(aviso_id: int, session: Session =
Depends(get_session)):
    aviso = session.get(Aviso, aviso_id)
    if not aviso:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"No se ha encontrado el aviso con ID {aviso_id}"
        )
    return aviso


# 4. ACTUALIZAR PARCIALMENTE UN AVISO (PATCH)
@app.patch("/avisos/{aviso_id}", response_model=Aviso)
def actualizar_aviso_protegido(
    aviso_id: int,
```

```
    aviso_update: AvisoUpdate,
    usuario_admin: str = Depends(obtener_usuario_admin_actual),
    session: Session = Depends(get_session)
):
    aviso_db = session.get(Aviso, aviso_id)
    if not aviso_db:
        raise HTTPException(status_code=404, detail="Aviso no
encontrado")

    datos = aviso_update.model_dump(exclude_unset=True)
    for clave, valor in datos.items():
        setattr(aviso_db, clave, valor)

    aviso_db.modificado_por = usuario_admin
    session.add(aviso_db)
    session.commit()
    session.refresh(aviso_db)
    return aviso_db


# 5. ELIMINAR UN AVISO (DELETE)
@app.delete("/avisos/{aviso_id}",
status_code=status.HTTP_204_NO_CONTENT)
def eliminar_aviso(
    aviso_id: int,
    usuario_admin: str = Depends(obtener_usuario_admin_actual),
    session: Session = Depends(get_session)
):
    aviso_db = session.get(Aviso, aviso_id)
    if not aviso_db:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail=f"No se ha encontrado el aviso con ID {aviso_id}"
        )

    session.delete(aviso_db)
    session.commit()
    return None
```

Tests de seguridad con `pytest`

El objeto `TestClient` permite pasar la tupla `auth=("usuario", "password")` para simular las cabeceras `HTTP Basic`.
Código del archivo de pruebas:

```
from datetime import datetime


# 1. Test de lectura pública (Sin necesidad de autenticación)
def test_lectura_avisos_publica(client):
    response = client.get("/avisos")
    assert response.status_code == 200


# 2. Test de intento de creación SIN credenciales (Debe fallar con
401)
def test_crear_aviso_sin_autenticacion_falla(client):
    payload = {
        "titulo": "Intento no autorizado",
        "contenido_md": "Contenido",
        "tipo_destino": "usuario",
        "destino": "jgarcia",
        "fecha_inicio": "2026-03-30T10:00:00Z"
    }
    response = client.post("/avisos", json=payload)
    assert response.status_code == 401
    assert "WWW-Authenticate" in response.headers


# 3. Test de intento de creación con CREDENCIALES INCORRECTAS (Debe
fallar con 401)
def test_crear_aviso_credenciales_erroneas_falla(client):
    payload = {
        "titulo": "Intento con clave falsa",
        "contenido_md": "Contenido",
        "tipo_destino": "usuario",
        "destino": "jgarcia",
        "fecha_inicio": "2026-03-30T10:00:00Z"
    }
    response = client.post(
        "/avisos",
        json=payload,
```

```
        auth=("admin_dom", "clave_incorrecta")
    )
    assert response.status_code == 401


# 4. Test de creación CON CREDENCIALES CORRECTAS (Debe tener éxito 201)
def test_crear_aviso_autenticado_exito(client):
    payload = {
        "titulo": "Aviso de Seguridad Crítico",
        "contenido_md": "Cambio de contraseñas de dominio obligatorias",
        "tipo_destino": "grupo",
        "destino": "Todos_Los_Usuarios",
        "fecha_inicio": "2026-03-30T08:00:00Z"
    }
    # Pasamos las credenciales correctas en el argumento 'auth'
    response = client.post(
        "/avisos",
        json=payload,
        auth=("admin_dom", "P@ssw0rd2026")
    )
    assert response.status_code == 201
    datos = response.json()
    assert datos["creado_por"] == "admin_dom"
```

Bloque 6 (3h): Sistema de logging en fichero y middleware

- Objetivos: los alumnos aprenderán a interceptar todas las peticiones HTTP que llegan a la API de FastAPI mediante Middlewares, capturando métricas clave como el tiempo de ejecución, direcciones IP de origen, códigos de respuesta y rutas solicitadas. Además, configuraremos el módulo estándar de Python logging para persistir esta información en archivos de registro (.log), esenciales para la auditoría de seguridad y diagnósticos en infraestructuras de red de dominio Active Directory.

¿Qué es un Middleware en FastAPI?

Una capa de software por la que pasa toda petición `HTTP` antes de llegar a la ruta objetivo, y por la que pasa toda respuesta antes de devolverse al cliente.

Casos de uso típicos:

- Medición de tiempos de respuesta (Latencia de la API).
- Registro e inspección de peticiones (Auditoría).
- Inyección de cabeceras de respuesta personalizadas (ej. `X-Process-Time`).
- Validación general de seguridad o `CORS`.

Estructura del Middleware (`call_next`):

```
@app.middleware("http")
async def mi_middleware(request: Request, call_next):
    # 1. Código que se ejecuta ANTES de que la ruta procese la petición
    response = await call_next(request)
    # 2. Código que se ejecuta DESPUÉS de que la ruta devuelva la respuesta
    return response
```

El Módulo `logging` de Python

- Por qué no usar `print()` en producción: `print()` no guarda histórico en disco, no añade marcas de tiempo, `timestamps`, no distingue la severidad del mensaje y causa problemas de rendimiento al sincronizar E/S en procesos asíncronos.
- Jerarquía de Niveles de Registro:
  1. `DEBUG`: Información detallada, orientada al desarrollador.
  2. `INFO`: Confirmación de que las cosas están funcionando como se esperaba (ej. peticiones HTTP `200 OK`).
  3. `WARNING`: Indicación de que algo inesperado ocurrió o un problema en el futuro cercano (ej. intentos de acceso fallidos `401`).
  4. `ERROR`: Debido a un problema más grave, el software no pudo realizar una función (ej. fallos en la base de datos `500`).
  5. `CRITICAL`: Error grave, el programa no puede continuar ejecutándose.
- Manejadores, `handlers` y formateadores, `Formatters`:

- `FileHandler`: Guarda la salida formateada en un fichero `.log`.
- `Formatter`: Define el aspecto del texto registrado (ej. `[2026-09-20 10:15:00] [INFO] [192.168.1.50] GET /avisos - 200 OK (2.35ms)`).

Configuración del `Logger`: `profesor\src\logger.py`

```
import logging
from pathlib import Path


# Crear el directorio de logs si no existe dentro de profesor/src/
LOG_DIR = Path(__file__).resolve().parent / "logs"
LOG_DIR.mkdir(exist_ok=True)

LOG_FILE = LOG_DIR / "scad.log"

# Configurar el Logger del sistema
logger = logging.getLogger("scad_auditoria")
logger.setLevel(logging.INFO)

# Formateador de texto estructurado para el archivo de logs
formatter = logging.Formatter(
    fmt="%(asctime)s | %(levelname)-8s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
)

# Manejador para guardar en fichero de texto
file_handler = logging.FileHandler(LOG_FILE, encoding="utf-8")
file_handler.setFormatter(formatter)

# Manejador adicional para mostrar también en consola durante el
desarrollo
console_handler = logging.StreamHandler()
console_handler.setFormatter(formatter)

# Evitar duplicar manejadores si el módulo se importa varias veces
if not logger.handlers:
    logger.addHandler(file_handler)
    logger.addHandler(console_handler)
```

Implementación del middleware de auditoría: `profesor\src\main.py`

Integramos el middleware en la aplicación para interceptar el tiempo de proceso, la `IP` origen, el método `HTTP`, la `URL` recibida y el estado de la respuesta.

```
import time

from fastapi import FastAPI, Request

from profesor.src.logger import logger


app = FastAPI(title="SCAD - Módulo de Auditoría y Logs",
version="0.6.0")


# MIDDLEWARE DE AUDITORÍA Y AUDIT LOG
@app.middleware("http")
async def registrar_auditoria_middleware(request: Request,
call_next):
    tiempo_inicio = time.perf_counter()

    # Extraer la IP del cliente (considerando proxies corporativos
si los hubiera)
    ip_cliente = request.client.host if request.client else
"Desconocida"
    metodo = request.method
    url = request.url.path

    # Procesar la petición pasando el control a la ruta destino
    try:
        response = await call_next(request)
        tiempo_proceso_ms = (time.perf_counter() - tiempo_inicio) *
1000
        codigo_estado = response.status_code

        # Registrar según la severidad del código de estado HTTP
        mensaje_log = f"IP: {ip_cliente} | {metodo} {url} | Estado:
{codigo_estado} | Tiempo: {tiempo_proceso_ms:.2f}ms"
```

```
        if codigo_estado < 400:
            logger.info(mensaje_log)
        elif 400 <= codigo_estado < 500:
            logger.warning(mensaje_log)
        else:
            logger.error(mensaje_log)

        # Inyectar el tiempo de proceso en las cabeceras de
respuesta HTTP
        response.headers["X-Process-Time-MS"] = f"
{tiempo_proceso_ms:.2f}"
        return response

    except Exception as exc:
        tiempo_proceso_ms = (time.perf_counter() - tiempo_inicio) *
1000
        logger.error(f"IP: {ip_cliente} | {metodo} {url} | EXCEPCIÓN
NO CONTROLADA: {str(exc)} | Tiempo: {tiempo_proceso_ms:.2f}ms")
        raise exc
```

Ejercicio práctico para el alumno

1. En tu espacio de trabajo (`alumno\src\`), crea un archivo `logger.py` que configure la salida de registros hacia la carpeta `alumno\src\logs\scad.log` .
2. En `alumno\src\main.py` , registra el middleware de `HTTP` `@app.middleware("http")` .
3. El middleware debe:
   - Medir el tiempo de ejecución exacto de cada petición.
   - Obtener la `IP` cliente (`request.client.host`) y el recurso consultado.
   - Añadir una cabecera de respuesta personalizada llamada `X-Process-Time-MS` .
   - Escribir en el log con severidad `INFO` para códigos 2xx/3xx y `WARNING` para errores 4xx (como un `401 Unauthorized` o `404 Not Found`).
4. Inicia la API con `uv run uvicorn main:app --reload` dentro de `alumno\src\` , realiza peticiones desde `Swagger UI` y verifica que la carpeta `logs\` se genera automáticamente con el fichero `scad.log` .

5. Escribe un test en `alumno\tests\test_logging.py` usando `pytest` que compruebe que la cabecera `X-Process-Time-MS` se incluye en las respuestas `HTTP` y que el fichero de logs registra los eventos.

Solución

Fichero de configuración de `logging`: `src\logger.py`

```
import logging
from pathlib import Path


# Crear el directorio de logs si no existe dentro de alumno/src/
LOG_DIR = Path(__file__).resolve().parent / "logs"
LOG_DIR.mkdir(exist_ok=True)

LOG_FILE = LOG_DIR / "scad.log"

# Configurar el Logger del sistema
logger = logging.getLogger("scad_auditoria")
logger.setLevel(logging.INFO)

# Formateador de texto estructurado para el archivo de logs
formatter = logging.Formatter(
    fmt="%(asctime)s | %(levelname)-8s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S"
)

# Manejador para guardar en fichero de texto
file_handler = logging.FileHandler(LOG_FILE, encoding="utf-8")
file_handler.setFormatter(formatter)

# Manejador adicional para mostrar también en consola durante el
desarrollo
console_handler = logging.StreamHandler()
console_handler.setFormatter(formatter)

# Evitar duplicar manejadores si el módulo se importa varias veces
if not logger.handlers:
    logger.addHandler(file_handler)
    logger.addHandler(console_handler)
```

Configuración de `midleware` : `src\main.py`

```
import time

from fastapi import FastAPI, Request

from alumno.src.logger import logger


app = FastAPI(title="SCAD - Módulo de Auditoría y Logs",
version="0.6.0")


# MIDDLEWARE DE AUDITORÍA Y AUDIT LOG
@app.middleware("http")
async def registrar_auditoria_middleware(request: Request,
call_next):
    tiempo_inicio = time.perf_counter()

    # Extraer la IP del cliente (considerando proxies corporativos
si los hubiera)
    ip_cliente = request.client.host if request.client else
"Desconocida"
    metodo = request.method
    url = request.url.path

    # Procesar la petición pasando el control a la ruta destino
    try:
        response = await call_next(request)
        tiempo_proceso_ms = (time.perf_counter() - tiempo_inicio) *
1000
        codigo_estado = response.status_code

        # Registrar según la severidad del código de estado HTTP
        mensaje_log = f"IP: {ip_cliente} | {metodo} {url} | Estado:
{codigo_estado} | Tiempo: {tiempo_proceso_ms:.2f}ms"

        if codigo_estado < 400:
            logger.info(mensaje_log)
        elif 400 <= codigo_estado < 500:
            logger.warning(mensaje_log)
```

```
        else:
            logger.error(mensaje_log)

        # Inyectar el tiempo de proceso en las cabeceras de
respuesta HTTP
        response.headers["X-Process-Time-MS"] = f"
{tiempo_proceso_ms:.2f}"
        return response

    except Exception as exc:
        tiempo_proceso_ms = (time.perf_counter() - tiempo_inicio) *
1000
        logger.error(f"IP: {ip_cliente} | {metodo} {url} | EXCEPCIÓN
NO CONTROLADA: {str(exc)} | Tiempo: {tiempo_proceso_ms:.2f}ms")
        raise exc
```

Tests de `middleware` y `logging` con `pytest`: `alumno\tests\test_logging.py`)

```
from pathlib import Path

def test_middleware_inyecta_cabecera_tiempo(client):
    """Verifica que el middleware añade la cabecera X-Process-Time-
MS a cualquier respuesta."""
    response = client.get("/avisos")
    assert response.status_code == 200
    assert "X-Process-Time-MS" in response.headers

    # Validar que el valor sea un número flotante parseable
    tiempo_ms = float(response.headers["X-Process-Time-MS"])
    assert tiempo_ms >= 0.0


def test_escribir_log_en_fichero(client):
    """Comprueba que al realizar peticiones HTTP, el archivo
scad.log se crea y se actualiza."""
    # Realizar una petición
    response = client.get("/avisos")
    assert response.status_code == 200

    # Ubicación esperada del fichero de log
```

```
    log_file = Path("alumno/src/logs/scad.log")

    # En el entorno de tests, comprobar que el archivo existe
    if log_file.exists():
        contenido = log_file.read_text(encoding="utf-8")
        assert "GET /avisos" in contenido
```

Día 4: `Frontend` dinámico con `Jinja2`, `HTMX` y `Markdown`

Bloque 7 (3h): Renderizado `Jinja2` y conversión de `Markdown` a `HTML`

- Objetivos: Crear el panel web de administración para redactar avisos en sintaxis `Markdown` y renderizarlos a HTML.

1. Renderizado en el servidor con `Jinja2`
   - Paso del objeto `Request`: `FastAPI` requiere que en el contexto de la plantilla Jinja2 se incluya siempre el objeto `request` de la petición actual (`{"request": request, ...}`) para la resolución interna de `URLs` y cabeceras.
   - Sintaxis de Jinja2:
     - Interpolación de variables: `{{ aviso.titulo }}`
     - Estructuras de control: `{% for aviso in avisos %}` y `{% if aviso.activo %}`
     - Herencia de diseño: `{% extends "base.html" %}` y `{% block contenido %}`
2. Transformación de `Markdown` a `HTML`
   - Procesamiento de contenido en el proyecto SCAD: Los avisos creados por los administradores admiten formato `Markdown` (negritas, listas, enlaces, código).
   - Filtros personalizados en `Jinja2`:
     - Permiten transformar un valor antes de volcarlo al `HTML`: `{{ aviso.contenido_md | markdown | safe }}`.
     - Por defecto, `Jinja2` escapa el `HTML` para prevenir ataques `XSS`. El modificador `| safe` indica al motor que el resultado del filtro `markdown` es `HTML` seguro que debe renderizarse directamente.

Código de ejemplo

Instalación de dependencias

```
uv add jinja2 markdown
```

Modificación de la estructura del proyecto

```
\tutorial
└── \profesor
    └── \src
        ├── \static
        │   └── \css
        │       └── styles.css
        ├── \templates
        │   ├── base.html
        │   └── avisos.html
        ├── auth.py
        ├── logger.py
        ├── main.py
        └── models.py
```

Definición del diseño base: `profesor\src\templates\base.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>{% block titulo %}SCAD - Sistema Centralizado de Avisos
de Dominio{% endblock %}</title>
    <link rel="stylesheet" href="/static/css/styles.css">
</head>
<body>
    <header>
        <h1>SCAD - Tablón de Avisos de Dominio</h1>
    </header>

    <main>
```

```
        {% block contenido %}{% endblock %}
    </main>

    <footer>
        <p>Sistema de Avisos Corporativo - Active Directory</p>
    </footer>
</body>
</html>
```

Vista principal de avisos: `profesor\src\templates\avisos.html`

```
{% extends "base.html" %}

{% block titulo %}Avisos Activos - SCAD{% endblock %}

{% block contenido %}
<h2>Avisos Vigentes</h2>

{% if avisos %}
    <div class="lista-avisos">
        {% for aviso in avisos %}
            <article class="tarjeta-aviso">
                <h3>{{ aviso.titulo }}</h3>
                <p class="meta">
                    Destino: <strong>{{ aviso.destino }}</strong> |
                    Publicado por: {{ aviso.creado_por }}
                </p>
                <div class="cuerpo-aviso">
                    {{ aviso.contenido_md | markdown | safe }}
                </div>
            </article>
        {% endfor %}
    </div>
{% else %}
    <p>No hay avisos activos para mostrar en este momento.</p>
{% endif %}
{% endblock %}
```

Configuración de `Jinja2` y filtros: `profesor\src\main.py`

```python
from pathlib import Path
from typing import Optional

from fastapi import FastAPI, Request, Depends, HTTPException, status
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
import markdown
from sqlmodel import Session, select

from frofesor.src.models import Aviso, get_session


# Path del archivo actual (profesor/src/main.py)
# .parent se sitúa en profesor/src/
BASE_DIR = Path(__file__).resolve().parent

# Verificación de seguridad de rutas
STATIC_DIR = BASE_DIR / "static"
TEMPLATES_DIR = BASE_DIR / "templates"

app = FastAPI(title="SCAD - Módulo HTML Jinja2", version="0.7.0")

# Montaje de archivos estáticos y plantillas
# Montaje con ruta absoluta garantizada
app.mount("/static", StaticFiles(directory=str(STATIC_DIR)),
name="static")
templates = Jinja2Templates(directory=str(TEMPLATES_DIR))


# Definición del filtro personalizado de Markdown
def filtro_markdown(texto: str) -> str:
    if not texto:
        return ""
    return markdown.markdown(
        texto,
        extensions=["fenced_code", "tables", "nl2br"]
    )

# Registrar el filtro en el entorno de Jinja2
templates.env.filters["markdown"] = filtro_markdown
```

```python
# RUTA WEB: Renderizado HTML de la lista de avisos
@app.get("/", response_class=HTMLResponse)
def mostrar_tablon_avisos(
    request: Request,
    destino: Optional[str] = None,
    session: Session = Depends(get_session)
):
    query = select(Aviso).where(Aviso.activo == True)
    if destino:
        query = query.where(Aviso.destino == destino)

    avisos = session.exec(query).all()

    # Se especifica "request" como primer parámetro posicional
    # y "context" como parámetro con nombre
    return templates.TemplateResponse(
        request=request,
        name="avisos.html",
        context={
            "avisos": avisos,
            "destino_actual": destino
        }
    )
```

Ejercicio práctico para el alumno

1. Instala las librerías `jinja2` y `markdown` en el entorno usando `uv add jinja2 markdown`.
2. Crea los directorios `alumno\src\templates\` y `alumno\src\static\css\`.
3. Crea el archivo `alumno\src\static\css\styles.css` con estilos básicos para las tarjetas de los avisos.
4. Implementa `base.html` y `avisos.html` dentro de `alumno\src\templates\`.
5. En `alumno\src\main.py`:
   - Configura `Jinja2Templates` apuntando a la carpeta de plantillas de `alumno\src\`.
   - Monta el directorio `static`.

- Registra el filtro `markdown` que convierte el texto en formato `Markdown` a `HTML`.
- Crea la ruta raíz `@app.get("/", response_class=HTMLResponse)` que devuelva la plantilla `avisos.html` con los avisos cargados desde la base de datos SQLite.

6. Arranca el servidor con `uv run uvicorn alumno.src.main:app --reload` y comprueba en el navegador en la `URL` `http://127.0.0.1:8000`, que el texto con formato `Markdown` se renderiza como `HTML`.

Test de renderizado

Veamos cómo utilizar `TestClient` para validar las respuestas `HTML`

```
def test_ruta_raiz_devuelve_html(client):
    """Verifica que la ruta principal devuelve un código 200 y
contenido HTML."""
    response = client.get("/")
    assert response.status_code == 200
    assert "text/html" in response.headers["content-type"]
    assert "SCAD - Tablón de Avisos" in response.text


from datetime import datetime, timezone

def test_renderizado_contenido_markdown(client):
    """Comprueba que el contenido en Markdown se convierte
correctamente a etiquetas HTML."""
    payload = {
        "titulo": "Aviso con Formato",
        "contenido_md": "**Atencion:** Servidor en *mantenimiento*",
        "tipo_destino": "grupo",
        "destino": "TODOS",
        "fecha_inicio": datetime.now(timezone.utc).isoformat(),
        "activo": True
    }

    # 1. Crear el aviso pasando la validación de Pydantic
    response_post = client.post("/avisos", json=payload, auth=
("admin_dom", "P@ssw0rd2026"))
    assert response_post.status_code in (200, 201), f"Error al crear
```

```
aviso: {response_post.json()}"

    # 2. Consultar la vista HTML
    response = client.get("/")
    assert response.status_code == 200

    # 3. Validar el renderizado del filtro Markdown
    assert "<strong>Atencion:</strong>" in response.text
    assert "<em>mantenimiento</em>" in response.text
```

Bloque 8 (3h): Interfaz reactiva con `HTMX`

- Objetivos: Construir el panel de administración dinámico sin escribir código JavaScript utilizando HTMX.

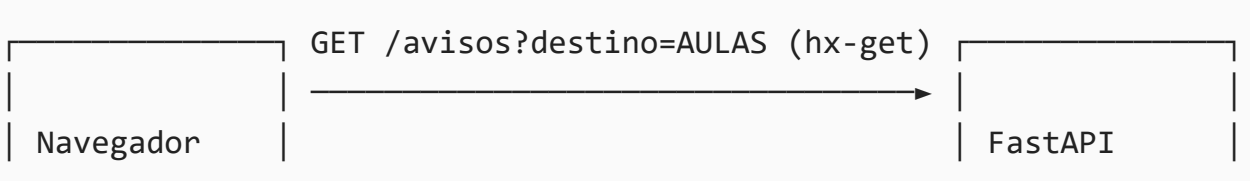
¿Qué es `HTMX` y cómo encaja con `FastAPI`?

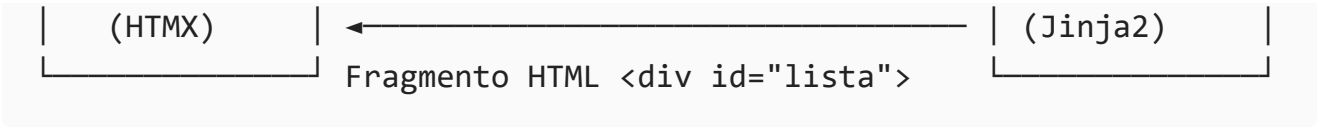
En la arquitectura web tradicional:

- Una petición `HTTP` normal devuelve páginas completas, lo que conlleva una redefinición total del DOM.
- Una `API REST` tradicional, en formato `JSON`, devuelve datos puros, requiriendo un `framework` cliente, como `React`, `Vue`, etc., para renderizarlos.

El enfoque `HTMX` + `FastAPI`:

- `HTMX` extiende la sintaxis de `HTML` permitiendo que cualquier elemento como un `<p>`, un `<div>` o un `<h1>`, realice peticiones `HTTP`, `GET`, `POST`, `DELETE`, de forma asíncrona.
- `FastAPI` procesa la petición y, en lugar de devolver la página completa o un `JSON`, devuelve un fragmento `HTML` con `Jinja2`.
- `HTMX` inyecta suavemente ese fragmento en la parte del DOM que le indiquemos.





Inclusión de `HTMX` en `base.html`

Añadiremos la librería `HTMX` desde una `CDN` limpia directamente en la plantilla base, por comodidad, aunque en producción la serviríamos desde nuestro servidor.

Actualizamos `profesor/src/templates/base.html` :

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>{% block titulo %}SCAD - Sistema Centralizado de Avisos
de Dominio{% endblock %}</title>
    <!-- Estilos base -->
    <link rel="stylesheet" href="static/css/styles.css">
    <!-- HTMX CDN -->
     <script
src="https://cdn.jsdelivr.net/npm/htmx.org@2.0.10/dist/htmx.min.js"
integrity="sha384-
H5SrcfygHmAuTDZphMHqBJLc3FhssKjG7w/CeCpFReSfwBWDTKpkzPP8c+cLsK+V"
crossorigin="anonymous"></script>
</head>
<body>
    <header>
        <h1>SCAD - Tablón de Avisos de Dominio</h1>
    </header>

    <main>
        {% block contenido %}{% endblock %}
    </main>

    <footer>
        <p>Sistema de Avisos Corporativo - Active Directory</p>
    </footer>
```

```
</body>
</html>
```

Modificación de plantillas para filtrado dinámico

Para que el filtrado por destino sea reactivo (sin recargar la página), crearemos dos plantillas

1. `_lista_avisos.html` : El fragmento dinámico que contiene únicamente las tarjetas.

```
<div id="contenedor-avisos" class="lista-avisos">
    {% if avisos %}
        {% for aviso in avisos %}
        <article class="tarjeta-aviso">
            <h3>{{ aviso.titulo }}</h3>
            <div class="meta">
                <span>Destino: <strong>{{ aviso.destino }}</strong>
({{ aviso.tipo_destino }})</span>
                <span> | Creado: {{ aviso.fecha_inicio.strftime('%Y-
%m-%d %H:%M') }}</span>
            </div>
            <div class="cuerpo-aviso">
                {{ aviso.contenido_md | markdown }}
            </div>
        </article>
        {% endfor %}
    {% else %}
        <p>No hay avisos activos para el filtro seleccionado.</p>
    {% endif %}
</div>
```

2. `avisos.html` : La página principal que incluye el selector y el contenedor del fragmento.

```
{% extends "base.html" %}

{% block titulo %}Avisos Activos - SCAD{% endblock %}

{% block contenido %}
```

```
<h2>Avisos Vigentes</h2>

<!-- Filtro HTMX -->
<div style="margin-bottom: 1.5rem;">
    <label for="filtro-destino">Filtrar por Destino (AD):</label>
    <select
        id="filtro-destino"
        name="destino"
        hx-get="/"
        hx-target="#contenedor-avisos"
        hx-swap="outerHTML"
        hx-trigger="change"
    >
        <option value="">-- Todos los destinos --</option>
        <option value="TODOS" {% if destino_actual == 'TODOS'
%}selected{% endif %}>TODOS</option>
        <option value="AULA_101" {% if destino_actual == 'AULA_101'
%}selected{% endif %}>AULA_101</option>
        <option value="AULA_102" {% if destino_actual == 'AULA_102'
%}selected{% endif %}>AULA_102</option>
        <option value="ALUMNOS_ESO" {% if destino_actual ==
'ALUMNOS_ESO' %}selected{% endif %}>ALUMNOS_ESO</option>
    </select>
</div>

<!-- Contenedor dinámico que será reemplazado por HTMX -->
{% include "_lista_avisos.html" %}

{% endblock %}
```

Atributos `HTMX`

- `hx-get="/"` : Realiza una petición `GET` a la ruta raíz pasando el valor del selector como parámetro (`?destino=VALOR`).
- `hx-target="#contenedor-avisos"` : Indica qué elemento del `DOM` debe actualizarse con la respuesta recibida.
- `hx-swap="outerHTML"` : Reemplaza completamente el elemento objetivo por el fragmento recibido.
- `hx-trigger="change"` : Se dispara automáticamente cuando el usuario cambia la opción seleccionada.

Actualización del controlador `FastAPI` `profesor\src\main.py`

Debemos detectar cuándo la petición procede de `HTMX` mediante el encabezado `HTTP` `HX-Request`:

- Si la petición viene de `HTMX`: Devolvemos solo el fragmento `_lista_avisos.html`.
- Si la petición no viene de `HTMX`: Devolvemos la plantilla completa `avisos.html`.

Actualiza la función `mostrar_tablon_avisos` en `profesor\src\main.py`:

```
from fastapi import FastAPI, Request, Depends, Header
from typing import Optional
from sqlmodel import Session, select
from models import Aviso, get_session

# ... (resto de configuración de app y templates)

@app.get("/", response_class=HTMLResponse)
def mostrar_tablon_avisos(
    request: Request,
    destino: Optional[str] = None,
    session: Session = Depends(get_session),
    hx_request: Optional[str] = Header(None, alias="HX-Request")  # 
Detecta si la petición es de HTMX
):
    query = select(Aviso).where(Aviso.activo == True)

    if destino and destino.strip() != "":
        query = query.where(Aviso.destino == destino)

    avisos = session.exec(query).all()

    context = {
        "request": request,
        "avisos": avisos,
        "destino_actual": destino
    }

    # Si la petición proviene de HTMX, renderizamos ÚNICAMENTE el
```

```
fragmento
    if hx_request:
        return templates.TemplateResponse(
            request=request,
            name="_lista_avisos.html",
            context=context
        )

    # Petición habitual del navegador: renderiza la plantilla
completa
    return templates.TemplateResponse(
        request=request,
        name="avisos.html",
        context=context
    )
```

Test automatizado de integración con `HTMX`

Añadiremos un test específico para verificar que `FastAPI` responde correctamente con el fragmento parcial cuando la petición incluye la cabecera `HX-Request`.

Crea/actualiza el archivo `profesor\tests\test_htmx.py`:

```
from datetime import datetime, timezone


def test_filtro_htmx_devuelve_fragmento_parcial(client):
    """Verifica que al enviar la cabecera HX-Request solo se
renderice el fragmento del HTML."""
    # 1. Crear avisos con distintos destinos
    payload1 = {
        "titulo": "Aviso General",
        "contenido_md": "Contenido 1",
        "tipo_destino": "grupo",
        "destino": "TODOS",
        "fecha_inicio": datetime.now(timezone.utc).isoformat(),
        "activo": True
    }
    payload2 = {
        "titulo": "Aviso Aula 101",
```

```
        "contenido_md": "Contenido 2",
        "tipo_destino": "equipo",
        "destino": "AULA_101",
        "fecha_inicio": datetime.now(timezone.utc).isoformat(),
        "activo": True
    }

    client.post("/avisos", json=payload1, auth=("admin_dom",
"P@ssw0rd2026"))
    client.post("/avisos", json=payload2, auth=("admin_dom",
"P@ssw0rd2026"))

    # 2. Petición HTMX filtrando por AULA_101
    headers = {"HX-Request": "true"}
    response = client.get("/?destino=AULA_101", headers=headers)

    assert response.status_code == 200
    # Debe incluir la tarjeta filtrada
    assert "Aviso Aula 101" in response.text
    # No debe incluir la estructura global completa (sin <html> o
<header>)
    assert "<!DOCTYPE html>" not in response.text
    assert "<header>" not in response.text
```

Día 5: Negocio de dominio, script `logon.cmd` e integración end-to-end

Bloque 9 (3h): Algoritmo de evaluación de destinatarios y filtro de vigencia

- Objetivos: Desarrollar la lógica de negocio que evalúa la identidad de la máquina/usuario y sus fechas para entregar únicamente los avisos aplicables.

Las Reglas de Negocio del Dominio

Para que un aviso sea devuelto a un equipo/usuario, debe cumplir **dos condiciones simultáneas**:

1. Filtro de Vigencia Temporal: Un aviso solo es válido si:
    1. `activo == True`.
    2. `fecha_inicio <= ahora` (el aviso ya ha comenzado).
    3. `fecha_fin is None` **O** `fecha_fin >= ahora` (el aviso no ha caducado).
2. Algoritmo de definición del destinatario
    1. Un cliente envía a la API dos, o tres identificadores, obtenidos del sistema operativo, al iniciar sesión:
        - `usuario`: El nombre de usuario en el dominio (ej. `jgarcia`).
        - `equipo`: El nombre de la máquina en el dominio (ej. `AULA-101-PC02`).
        - `grupos`: La lista de grupos de AD a los que pertenece el usuario (ej. `["Docentes", "Usuarios del Dominio"]`).
    2. Un aviso debe ser enviado si su destino coincide con cualquiera de estas reglas

tipo_destino	destino del aviso	Condición de coincidencia
todos	cualquiera	Siempre coincide con cualquier usuario/equipo.
usuario	"jgarcia"	Coincide si el usuario solicitante es exactamente ese.
equipo	"AULA-101-PC02"	Coincide si el equipo solicitante es exactamente ese.
grupo	"Docentes"	Coincide si "Docentes" está dentro de la lista grupos del usuario.

Implementación de la lógica de negocio

Vamos a crear un módulo dedicado a la lógica de negocio en `profesor/src/evaluador.py`, o en una carpeta de servicios/dominio, para mantener los `endpoints` de `FastAPI` limpios

Crea el archivo `profesor\src\evaluador.py`:

```python
from datetime import datetime, UTC
from typing import List, Optional

from sqlmodel import Session, select, or_, and_

from profesor.src.models import Aviso  # O la ruta donde tengas tu modelo SQLModel


def obtener_avisos_vigentes_para_cliente(
    session: Session,
    usuario: str,
    equipo: str,
    grupos: Optional[List[str]] = None,
    momento: Optional[datetime] = None,
) -> List[Aviso]:
    """
    Evalúa y devuelve la lista de avisos activos y vigentes aplicables a un
    usuario, equipo y conjunto de grupos de Active Directory específicos.
    """
    if momento is None:
        momento = datetime.now(UTC)

    if grupos is None:
        grupos = []

    # Normalizamos a mayúsculas/minúsculas según convenga para evitar fallos de coincidencia
    usuario_clean = usuario.strip().lower()
    equipo_clean = equipo.strip().upper()
    grupos_clean = [g.strip().lower() for g in grupos]

    # 1. Cargar avisos activos con filtro de vigencia temporal
    query = select(Aviso).where(
        Aviso.activo == True,
        Aviso.fecha_inicio <= momento,
        or_(Aviso.fecha_fin == None, Aviso.fecha_fin >= momento),
    )
```

```
    candidatos = session.exec(query).all()

    # 2. Filtrar en memoria por algoritmo de destinatario
    avisos_evaluados = []

    for aviso in candidatos:
        tipo = aviso.tipo_destino.lower()
        destino = aviso.destino.strip()

        if tipo == "todos" or destino.upper() == "TODOS":
            avisos_evaluados.append(aviso)

        elif tipo == "usuario" and destino.lower() == usuario_clean:
            avisos_evaluados.append(aviso)

        elif tipo == "equipo" and destino.upper() == equipo_clean:
            avisos_evaluados.append(aviso)

        elif tipo == "grupo" and destino.lower() in grupos_clean:
            avisos_evaluados.append(aviso)

    return avisos_evaluados
```

Pruebas

Para asegurarnos de que el algoritmo no falle nunca ante casos de borde, avisos caducados, mayúsculas/minúsculas, pertenencia a grupos, añadiremos un archivo de tests específico en `profesor\tests\test_evaluador.py`

```
from datetime import datetime, timedelta, timezone

from sqlmodel import Session

from profesor.src.models import Aviso
from profesor.src.evaluador import
obtener_avisos_vigentes_para_cliente


def test_evaluacion_vigencia_y_destinatarios(session: Session):
    ahora = datetime.now(timezone.utc)
```

```
    # 1. Aviso General (Vigente)
    a_general = Aviso(
        titulo="Aviso General",
        contenido_md="Hola a todos",
        tipo_destino="todos",
        destino="TODOS",
        fecha_inicio=ahora - timedelta(hours=1),
        activo=True,
        creado_por="admin",
    )

    # 2. Aviso Caducado (No debe aparecer)
    a_caducado = Aviso(
        titulo="Aviso Antiguo",
        contenido_md="Expirado",
        tipo_destino="todos",
        destino="TODOS",
        fecha_inicio=ahora - timedelta(days=2),
        fecha_fin=ahora - timedelta(days=1),
        activo=True,
        creado_por="admin",
    )

    # 3. Aviso Específico para Grupo "Docentes"
    a_grupo = Aviso(
        titulo="Reunión Claustro",
        contenido_md="Aviso para profesores",
        tipo_destino="grupo",
        destino="Docentes",
        fecha_inicio=ahora - timedelta(minutes=10),
        activo=True,
        creado_por="admin",
    )

    # 4. Aviso para otro equipo
    a_equipo_otro = Aviso(
        titulo="Mantenimiento Aula 200",
        contenido_md="Solo Aula 200",
        tipo_destino="equipo",
        destino="AULA-200-PC01",
        fecha_inicio=ahora - timedelta(minutes=10),
```

```
        activo=True,
        creado_por="admin",
    )

    session.add_all([a_general, a_caducado, a_grupo, a_equipo_otro])
    session.commit()

    # EJECUCIÓN: Petición desde un PC del Aula 101 por un usuario
del grupo "Docentes"
    resultado = obtener_avisos_vigentes_para_cliente(
        session=session,
        usuario="jgarcia",
        equipo="AULA-101-PC05",
        grupos=["docentes", "usuarios del dominio"],
        momento=ahora,
    )

    titulos_obtenidos = [a.titulo for a in resultado]

    # COMPROBACIONES
    assert "Aviso General" in titulos_obtenidos
    assert "Reunión Claustro" in titulos_obtenidos
    assert "Aviso Antiguo" not in titulos_obtenidos
    assert "Mantenimiento Aula 200" not in titulos_obtenidos
    assert len(resultado) == 2
```

- Ejercicio práctico: Escribir un archivo de pruebas `tests/test_negocio.py` con `pytest` comprobando mediante un escenario parametrizado que un usuario recibe sus avisos directos, de su grupo y de su OU, ignorando los destinados a otros equipos.

Bloque 10 (3h): Agente cliente (`logon.cmd`), pruebas integrales E2E y cierre

- Objetivos: Conectar el script de inicio de sesión de Windows con la API, ejecutar la suite completa de pruebas y finalizar el proyecto.

Llegamos al punto donde convergen todas las piezas: el `endpoint` en `FastAPI` que recibe el `payload` enviado por el `logon.cmd` , evalúa la vigencia

y el destino de los avisos disponibles en la base de datos, y responde indicando si hay avisos junto con la URL del portal web para abrirlos.

Esquemas `Pydantic` para la petición y respuesta

Primero definimos la estructura del `JSON` que envía el cliente y la respuesta que espera recibir para abrir el navegador.
Crea o actualiza el archivo `profesor/src/schemas.py`:

```
from typing import List, Optional

from pydantic import BaseModel


class PeticionDeLogon(BaseModel):
    """Payload enviado por el logoncmd al iniciar sesión."""

    usuario: str
    equipo: str
    grupos: List[str] = []


class RespuestaParaLogon(BaseModel):
    """Respuesta enviada de vuelta al logoncmd."""

    hay_avisos: bool
    total_avisos: int
    url_portal: Optional[str] = None
```

Creación de `endpoint` en `FastAPI`

Este `endpoint` procesa la solicitud del cliente. Para garantizar que cada sesión de avisos sea segura y no se puedan manipular los parámetros en la URL, generamos un token temporal firmado (`JWT` o `UUID` de sesión) o pasamos los identificadores codificados de forma segura para rendir la página de tarjetas web.

Añade la ruta en tu archivo de API `profesor/src/routes/main_avisos.py`:

```python
import base64
import json
from pathlib import Path
import sys
import traceback
from typing import List

from fastapi import FastAPI, Depends, Request
from fastapi.responses import HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
import markdown2
from sqlmodel import Session

from profesor.src.models import get_session
from profesor.src.evaluador import
obtener_avisos_vigentes_para_cliente
from profesor.src.schemas import PeticionDeLogon, RespuestaParaLogon


def md_to_html(text: str) -> str:
    return markdown2.markdown(text)


# Path del archivo actual (profesor/src/main.py)
# .parent se sitúa en profesor/src/
BASE_DIR = Path(__file__).resolve().parent

# Verificación de seguridad de rutas
STATIC_DIR = BASE_DIR / "static"
TEMPLATES_DIR = BASE_DIR / "templates"


app = FastAPI(title="SCAD ", version="0.10.0")


# Controlador global para imprimir la traza exacta en la consola
@app.exception_handler(Exception)
async def debug_exception_handler(request: Request, exc: Exception):
    # 1. Escribir la traza en la consola de errores y FORZAR el
vaciado del buffer (flush)
```

```
    sys.stderr.write("\n" + "=" * 80 + "\n")
    sys.stderr.write(f" EXCEPCIÓN EN PETICIÓN: {request.method}
{request.url.path}\n")
    sys.stderr.write("=" * 80 + "\n")
    traceback.print_exc(file=sys.stderr)
    sys.stderr.write("=" * 80 + "\n\n")
    sys.stderr.flush()  # <-- Esto obliga a Uvicorn a pintar
inmediatamente en la terminal

    return JSONResponse(
        status_code=500,
        content={
            "error": str(exc),
            "type": type(exc).__name__,
            "message": "Revisa la terminal para ver el traceback
completo.",
            "path": request.url.path,
        },
    )


# Montar los archivos estáticos (CSS, JS, imágenes)
app.mount("/static", StaticFiles(directory=STATIC_DIR),
name="static")

templates = Jinja2Templates(directory=TEMPLATES_DIR)


@app.post("/api/v1/evaluar-avisos",
response_model=RespuestaParaLogon)
def evaluar_avisos_cliente(
    payload: PeticionDeLogon,
    request: Request,
    session: Session = Depends(get_session),
):
    """Endpoint invocado por el script de inicio de sesión
(AgenteSCAD.ps1).

    Evalúa qué avisos le corresponden al usuario/equipo y devuelve
la URL para
    su visualización en el navegador.
    """
```

```
    print(f'{payload=}')

    # 1. Obtener los avisos aplicables utilizando el motor de
evaluación
    avisos_vigentes = obtener_avisos_vigentes_para_cliente(
        session=session,
        usuario=payload.usuario,
        equipo=payload.equipo,
        grupos=payload.grupos,
    )

    if not avisos_vigentes:
        return RespuestaParaLogon(hay_avisos=False, total_avisos=0)

    # 2. Codificar la identidad de la consulta en un token ligero
para la URL
    # (En producción esto se firma con JWT/itsdangerous para evitar
manipulaciones)
    token_data = {
        "usuario": payload.usuario,
        "equipo": payload.equipo,
        "grupos": payload.grupos,
    }
    token_encoded = base64.urlsafe_b64encode(
        json.dumps(token_data).encode()
    ).decode()

    # 3. Construir la URL completa apuntando al portal de
renderizado HTML
    base_url = str(request.base_url).rstrip("/")
    url_portal = f"{base_url}/portal-avisos?session_token=
{token_encoded}"

    return RespuestaParaLogon(
        hay_avisos=True, total_avisos=len(avisos_vigentes),
url_portal=url_portal
    )
```

`Endpoint` del portal web

Esta es la ruta que se abrirá en el navegador por defecto del usuario cuando `logon.cm` ejecute `Start-Process $respuesta.url_portal`.

```
@app.get("/portal-avisos", response_class=HTMLResponse)
def mostrar_portal_avisos(
    session_token: str, request: Request, session: Session =
Depends(get_session)
):
    """Ruta que abre el navegador del usuario.

    Decodifica el token de sesión, re-evalúa los avisos y los
renderiza en la
    plantilla de tarjetas HTML.
    """
    try:
        # Decodificamos el token de la URL
        raw_json =
base64.urlsafe_b64decode(session_token.encode()).decode()
        datos_sesion = json.loads(raw_json)
    except Exception:
        return HTMLResponse(
            content="<h3>Error de sesión inválida</h3>",
status_code=400
        )

    usuario = datos_sesion.get("usuario", "")
    equipo = datos_sesion.get("equipo", "")
    grupos = datos_sesion.get("grupos", [])

    # Re-evaluamos los avisos para asegurar que están vigentes en el
instante exacto de carga
    avisos_vigentes = obtener_avisos_vigentes_para_cliente(
        session=session, usuario=usuario, equipo=equipo,
grupos=grupos
    )

    # Convertimos el contenido Markdown de cada aviso a HTML
    avisos_renderizados = []
    for aviso in avisos_vigentes:
        aviso_dict = {
            "id": aviso.id,
```

```
            "titulo": aviso.titulo,
            "contenido_html": md_to_html(aviso.contenido_md),
            "importancia": (
                aviso.importancia if hasattr(aviso, "importancia")
else "media"
            ),
            "fecha_inicio": aviso.fecha_inicio,
        }
        avisos_renderizados.append(aviso_dict)

    return templates.TemplateResponse(
        "portal_avisos.html",
        {
            "request": request,
            "usuario": usuario,
            "equipo": equipo,
            "avisos": avisos_renderizados,
        },
    )
```

Plantilla `profesor/src/templates/portal_avisos.html`

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Portal de Avisos - SCAD</title>
    <!-- Hoja de Estilos Unificada SCAD -->
    <link rel="stylesheet" href="/static/css/scad.css">
</head>
<body>
    <header>
        <div class="header-container">
            <h1>📢 SCAD Portal</h1>
            <div class="usuario-tag">
                Usuario: <strong>{{ usuario }}</strong> {% if equipo
%}({{ equipo }}){% endif %}
            </div>
        </div>
```

```
    </header>

    <main>
        <h2>Notificaciones Vigentes</h2>

        {% if avisos %}
            <div class="lista-avisos">
                {% for aviso in avisos %}
                <article class="tarjeta-aviso {{ aviso.importancia |
lower }}">
                    <div class="tarjeta-header">
                        <h3>{{ aviso.titulo }}</h3>
                        {% if aviso.importancia %}
                        <span class="badge {{ aviso.importancia |
lower }}">{{ aviso.importancia }}</span>
                        {% endif %}
                    </div>

                    <div class="meta">
                        Publicado: <strong>{{
aviso.fecha_inicio.strftime('%d/%m/%Y %H:%M') if aviso.fecha_inicio
else 'Reciente' }}</strong>
                    </div>

                    <!-- Renderizado seguro de HTML generado desde
Markdown -->
                    <div class="cuerpo-aviso">
                        {{ aviso.contenido_html | safe }}
                    </div>
                </article>
                {% endfor %}
            </div>
        {% else %}
            <div class="estado-vacio">
                <h3>Sin avisos pendientes</h3>
                <p>No hay notificaciones activas asignadas a tu
cuenta o equipo en este momento.</p>
            </div>
        {% endif %}
    </main>

    <footer>
```

```
        SCAD &copy; 2026 - Sistema Centralizado de Avisos de Dominio
    </footer>
</body>
</html>
```

Test de integración `End-to-End`

Validamos todo el flujo completo mediante un test con `TestClient` de `FastAPI`: simula la llamada del script en el inicio de sesión y la posterior apertura del navegador.

Crea el archivo `profesor/tests/test_e2e_logon.py`:

```
from datetime import datetime, timedelta, timezone
from fastapi.testclient import TestClient
from sqlmodel import Session

from profesor.src.models import Aviso


def test_flujo_completo_logon_a_portal_web(client: TestClient,
session: Session):
    ahora = datetime.now(timezone.utc)

    # 1. Crear avisos de prueba en la BD
    aviso_alumnos = Aviso(
        titulo="Mantenimiento de Red",
        contenido_md="El WiFi se cortará a las **17:00h**. Más info
[aquí](https://ejemplo.local).",
        tipo_destino="grupo",
        destino="Alumnos",
        fecha_inicio=ahora - timedelta(minutes=5),
        activo=True,
        creado_por="admin",
    )
    session.add(aviso_alumnos)
    session.commit()

    # 2. PASO 1: El Agente Client hace POST enviando la identidad
    payload = {
        "usuario": "perez_m",
```

```
        "equipo": "AULA-01-PC02",
        "grupos": ["Alumnos", "Usuarios del Dominio"],
    }

    response_logon = client.post("/api/v1/evaluar-avisos",
json=payload)
    assert response_logon.status_code == 200

    data = response_logon.json()
    assert data["hay_avisos"] is True
    assert data["total_avisos"] == 1
    assert "url_portal" in data

    # 3. PASO 2: El Navegador abre la URL devuelta en el paso
anterior
    url_portal = data["url_portal"]
    # Extraemos la ruta relativa para el TestClient (/portal-avisos?
session_token=...)
    path_portal = url_portal.split("http://testserver")[-1]

    response_portal = client.get(path_portal)
    assert response_portal.status_code == 200
    assert "text/html" in response_portal.headers["content-type"]

    # Validamos que el HTML renderizado contenga las tarjetas y el
Markdown transformado
    html = response_portal.text
    assert "Mantenimiento de Red" in html
    assert "<strong>17:00h</strong>" in html or "<b>17:00h</b>" in
html
    assert 'href="https://ejemplo.local"' in html
```

Ejecuta el test para certificar la integración E2E:

```
uv run pytest -v -s ./profesor/tests/test_e2e_logon.py
```

Con esto queda completamente cerrado el pipeline: el cliente consulta sus credenciales locales en el logon, el servidor evalúa y responde con la URL, y el navegador despliega la interfaz con las tarjetas formateadas en HTML.